

“Flight Status Tracker Management”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF ELECTRONIC COMMUNICATION AND ENGINEERING

&

DEPARTMENT OF ELECTRONIC COMMUNICATION AND ENGINEERING

By

THUNGA SREE CHANDANA-2520040030

NARALASETTY HASINI- 2520040039

Under the Esteemed Guidance of

DR KOTTE ANUSHA

Assistant Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF ELECTRONIC COMMUNICATION AND ENGINEERING
&
DEPARTMENT OF ELECTRONIC COMMUNICATION AND ENGINEERING



Declaration

The Project Report entitled **“FLIGHT STATUS TRACKER MANAGEMENT”** is a record of Bonafide work of **THUNGA SREE CHANDANA-2520040030, NARALASETTY HASINI-2520040039** submitted in partial fulfillment for the award of B. Tech in Electronic Communication and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

T.SREE CHANDANA-2520040030

NARALASETTY HASINI-2520040039

K L (Deemed to be) University

DEPARTMENT OF ELECTRONIC COMMUNICATION AND ENGINEERING

&

DEPARTMENT OF ELECTRONIC COMMUNICATION AND ENGINEERING



CERTIFICATE

This is certify that the mini project based report entitled **“INTELLIGENT NAVIGATION AND SAFETY PLATFORM”** is a bonafide work done and submitted by **THUNGA SREE CHANDANA-2520040030 NARALASETTY HASINI-2520040039**

in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Electronic Communication And Engineering, K L (Deemed to be University), during the academic year **2025-2026**.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator **Dr. K. MADHAVI REDDY**, and my guide **Dr. K. ANUSHA** Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Sunkara Srinivasa Rao (Ece)**, Head of the **Department of Electronic communication and Engineering** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

THUNGA SREE CHANDANA-2520040030

NARALASETTY HASINI -2520040039

Abstract

The Flight Status Tracker is a web-based application developed to provide passengers and airline personnel with accurate and real-time flight information through a simple and user-friendly interface. The main objective of the system is to help users easily monitor flight schedules and stay informed about any changes in flight status. The application allows users to search for flights using the flight number and view important details such as the airline name, departure and arrival airports, scheduled and estimated departure and arrival times, gate information, terminal details, and the current flight status, including On Time, Delayed, Boarding, Departed, Arrived, or Cancelled. The system integrates REST APIs to fetch live flight data from external sources and uses polling techniques to automatically refresh the displayed information at regular intervals, ensuring that users always receive the latest updates without manually reloading the page. Users can also subscribe to notifications for selected flights and receive alerts regarding delays, gate changes, cancellations, or schedule modifications, allowing them to plan their journey more effectively.

The application is developed using React.js for the frontend and Node.js for the backend, with optional MySQL integration for storing user preferences, notification subscriptions, and historical flight data. Additional features include a flight status dashboard, airline-based filtering, responsive design for different devices, a map view for flight tracking (where supported), and an admin panel for managing users and monitoring the system. Overall, the Flight Status Tracker provides a reliable, efficient, and user-friendly platform for monitoring flight information in real time while demonstrating the use of modern web technologies, API integration, and full-stack development concepts.

Index

S. No.	Chapters	Topics	Page.no
		Acknowledgement	
		Abstract	
1	Introduction	1.1 Background of the project 1.2 Problem statement 1.3 Scope of the project 1.4 Objective(s) 1.5 Importance of the application 1.6 Target users/audience	5-14
2	System Requirements	2.1 Hardware requirements 2.2 Software requirements 2.3 Development tools and frameworks	15-21
3	Technology Stack	3.1 Front-end (e.g., React.js, Angular, HTML/CSS) 3.2 Back-end (e.g., Node.js, Django, Spring Boot) 3.3 Database (e.g., MongoDB, MySQL, PostgreSQL) 3.4 Version Control (e.g., Git, GitHub) 3.5 APIs or External services (e.g., Firebase, Stripe)	22-28
4	System Architecture	4.1 High-level architecture diagram 4.2 Description of each layer (frontend, backend, database) 4.3 Deployment architecture (e.g., cloud, local, CI/CD pipelines)	29-34
5	Design	5.2 ER Diagram	35
6	Implementation	6.1 Module-wise implementation 6.2 Front-end logic (UI rendering, state management) 6.3 API integration 6.4 Authentication & Authorization	38-42
7	Features	7.1 List of main features 7.2 How each feature works (user perspective + technical description)	43-46
8	Testing	8.1 Postman testing (frontend/backend) 8.2 Integration testing 8.3 Tools used for testing 8.4 Test cases and results	47-56
9	Deployment	9.1 Steps to deploy 9.2 Environment configuration 9.3 Hosting of frontend and backend	57-64
10	Challenges & Limitations	10.1 Issues faced during development 10.2 Solutions applied 10.3 Current limitations or known bugs	65-73
11	Future Enhancements	11.1 Planned features 11.2 Possible integrations or optimizations	74-79
12	Conclusion	12.1 Summary of the project 12.2 What was achieved 12.3 Skills learned during development	80-86
13	References	- Books, tutorials, APIs, documentation sites used	87-88

14	Appendices	- Screenshots of the app - Sample code snippets - Installation/setup instructions - User manual or guide -Geo Tag photos with guide	89-95
----	------------	---	-------

CHAPTER -1 INTRODUCTION

The aviation industry has experienced significant growth over the past few decades, with millions of passengers traveling by air every day for business, tourism, education, and personal purposes. As air travel continues to increase, the demand for accurate and real-time flight information has become more important than ever. Passengers, airline staff, and airport authorities rely on timely flight updates to manage travel schedules, reduce delays, and improve overall operational efficiency. Access to reliable flight information helps passengers plan their journeys more effectively and minimizes confusion caused by unexpected schedule changes.

Traditionally, passengers obtained flight information through airport display boards, airline customer service desks, or telephone inquiries. Although these methods provide basic information, they often require passengers to be physically present at the airport or depend on manual communication, which may not always provide the latest updates. Delays, gate changes, cancellations, and schedule modifications can occur at any time, making it essential to have a system that delivers real-time flight information directly to users.

With the advancement of web technologies, cloud services, and RESTful APIs, it has become possible to develop intelligent flight tracking systems that provide live updates through web applications. Modern flight information systems can retrieve real-time data from aviation service providers and present it in an easy-to-understand format. These technologies allow users to monitor flights from anywhere using a computer or mobile device, eliminating the need to rely solely on airport information displays.

The Flight Status Tracker is developed to provide a simple, reliable, and user-friendly platform for monitoring flight information in real time. The application allows users to search for flights using flight numbers and view important details such as airline information, departure and arrival airports, scheduled and estimated timings, gate information, and current flight status. The system integrates REST APIs to fetch live flight data and uses polling techniques to automatically refresh the information at regular intervals, ensuring that users always receive the most recent updates.

In addition to flight tracking, the application enables users to subscribe to notifications for selected flights and receive alerts regarding delays, cancellations, gate changes, or schedule modifications. The system also includes features such as a flight status dashboard, airline-based filtering, responsive design for multiple devices, a map view for supported flights, and an admin panel for managing users and monitoring application data. These features enhance user convenience while providing an efficient platform for accessing flight information.

The project is developed using modern web technologies including **React.js** for the frontend and **Node.js** for the backend, with optional **MySQL** integration for storing user preferences, notification subscriptions, and historical flight data. The application follows a RESTful architecture, enabling efficient communication between the frontend, backend, database, and external flight data services.

As airports and airlines continue to adopt digital technologies to improve customer experience, real-time flight tracking systems have become an essential part of modern air travel. The Flight Status Tracker addresses this need by providing accurate, up-to-date, and easily accessible flight information through a single web platform. The project demonstrates how modern full-stack web development,

REST API integration, and real-time data synchronization can be combined to build an efficient and scalable flight information system that benefits passengers, airlines, and airport authorities.

1.2 Problem Statement

The rapid growth of the aviation industry and the increasing number of air travelers have created a strong demand for accurate, reliable, and real-time flight information. Passengers, airline staff, airport authorities, and travel agencies require up-to-date flight details to plan their activities efficiently and avoid inconvenience caused by unexpected schedule changes. Flight delays, cancellations, gate changes, and other operational updates occur frequently, making it essential for users to have instant access to current flight information.

Traditional methods of obtaining flight information, such as airport display boards, airline service counters, telephone inquiries, and manually refreshing airline websites, are often time-consuming and may not always provide the latest updates. Passengers traveling to or from airports may find it difficult to continuously monitor changes in flight schedules, leading to confusion, missed flights, unnecessary waiting times, and poor travel experiences. The lack of timely information can also affect family members waiting to receive passengers and airport staff responsible for managing flight operations.

Although many airline websites and mobile applications provide flight status information, they are generally limited to specific airlines and may not offer a centralized platform for tracking flights from multiple airlines. In addition, some systems do not provide real-time automatic updates or personalized notification services, requiring users to repeatedly check flight information manually. This creates inconvenience, especially when flight schedules change frequently due to weather conditions, technical issues, or operational delays.

To overcome these challenges, the Flight Status Tracker is proposed as a web-based application that provides real-time flight information through a simple, interactive, and user-friendly platform. The system enables users to search for flights using flight numbers, view live flight details, receive automatic updates through REST API integration and polling techniques, and subscribe to notifications for important events such as delays, cancellations, gate changes, and schedule modifications. The application also includes features such as a flight status dashboard, airline-based filtering, responsive design, and an admin panel for efficient system management. By integrating these features into a single platform, the Flight Status Tracker aims to improve the accessibility, accuracy, and convenience of flight information for passengers and airline personnel.

Problems Identified

The major problems identified in the existing system are:

- Lack of a centralized platform for tracking flights from multiple airlines.

- Difficulty in obtaining real-time flight status updates.

- Frequent delays and schedule changes causing inconvenience to passengers.

- Dependence on manually refreshing websites or checking airport display boards.

- Delay in receiving information about gate changes and flight cancellations.

- Limited notification services for important flight updates.

- Inability to monitor multiple flights efficiently from a single dashboard.

- Lack of historical flight data for analysis and reference.

- Limited accessibility across different devices in some existing systems.

- Need for a user-friendly and responsive platform that provides reliable and real-time flight information.

The proposed Flight Status Tracker is designed to overcome these limitations by providing a centralized, efficient, and real-time flight monitoring system that enhances the travel experience for passengers while improving access to accurate flight information.

1.3 Scope of the Project

The scope of the **Flight Status Tracker** is focused on providing an efficient, reliable, and user-friendly platform that enables passengers, airline personnel, airport staff, and other users to monitor real-time flight information. The system is designed to simplify flight tracking by providing accurate and up-to-date flight details through a centralized web application. It helps users stay informed about flight schedules, delays, gate changes, cancellations, and other important flight-related updates.

The project covers the development of a web-based application that integrates live flight data using REST APIs, real-time status updates through polling techniques, user notifications, and flight management functionalities into a single platform. The application allows users to search for flights using flight numbers and obtain complete flight information through an interactive and responsive interface.

One of the major areas within the scope of the project is real-time flight tracking. The system enables users to search for flights and view important details such as airline name, departure and arrival airports, scheduled and estimated departure and arrival times, gate information, terminal details, and current flight status. By integrating live aviation data services, the application ensures that users always receive accurate and updated flight information.

Another important aspect of the project is the notification system. Registered users can subscribe to specific flights and receive alerts regarding delays, cancellations, gate changes, boarding updates, or schedule modifications. This feature helps passengers make informed travel decisions and reduces the inconvenience caused by unexpected flight changes.

The project also includes a flight status dashboard that displays multiple flights simultaneously, allowing users to monitor several flights from a single interface. Additional functionalities such as airline-based filtering, flight status filtering, search history, responsive design, and an optional map view enhance the overall usability of the application.

The Flight Status Tracker provides an administrative panel that enables authorized users to manage flight records, monitor user activities, maintain historical flight information, and manage notification services. This centralized management approach improves the efficiency of maintaining and monitoring the application.

The project is developed using modern web technologies including React.js for the frontend, Node.js for the backend, and optional MySQL integration for storing user preferences, notification subscriptions, and historical flight data. These technologies provide a responsive, scalable, and interactive user experience while ensuring efficient communication with external flight information services.

The scope of the project includes the following functionalities:

- User authentication and login management.
- Flight search using flight numbers.
- Real-time flight status tracking.
- Display of departure and arrival information.
- Gate, terminal, and schedule information.
- Automatic data updates using polling techniques.
- Flight notification and alert services.
- Flight status dashboard.
- Airline-based and status-based filtering.
- Historical flight data management.
- Administrative control panel.
- Responsive and user-friendly web interface.
- REST API integration for live flight information.

The system is intended for use by passengers, airline staff, airport authorities, travel agencies, and anyone who needs quick access to accurate flight information. It provides a centralized platform that improves accessibility, reduces

uncertainty caused by flight schedule changes, and enhances the overall travel experience.

Limitations of Current Scope

Although the current system provides essential flight tracking functionalities, certain features are beyond the scope of the present implementation. These include:

- Mobile application development.
- AI-based flight delay prediction.
- Integration with multiple international aviation databases.
- Online flight ticket booking and payment services.
- Boarding pass generation.
- SMS and email notification services.
- Weather-based flight prediction and analysis.
- Airport navigation and indoor maps.
- Cloud-based data storage and analytics.

These features can be incorporated in future versions to further enhance the capabilities of the Flight Status Tracker.

In summary, the scope of the Flight Status Tracker is to provide a comprehensive web-based solution that enables users to monitor flights efficiently through real-time updates, live notifications, and an interactive dashboard. By integrating modern web technologies, REST APIs, and responsive design, the system aims to improve the accessibility, reliability, and convenience of flight information while enhancing the overall travel experience.

1.4 Objectives of the Project

The primary objective of the Flight Status Tracker is to develop a reliable, user-friendly, and web-based platform that enables passengers, airline personnel, airport staff, and travel agencies to access accurate and real-time flight information. The system aims to provide live flight status updates by integrating REST APIs and real-time data synchronization techniques, allowing users to monitor flights efficiently and make informed travel decisions.

The project is designed to overcome the limitations of traditional flight information systems by providing instant flight updates, automated notifications, centralized flight tracking, and an interactive dashboard. Using modern web technologies, the application offers a responsive and efficient platform for monitoring flight schedules, reducing uncertainty caused by delays, cancellations, and other operational changes.

General Objective

To design and develop a web-based Flight Status Tracker that provides real-time flight information, live status updates, and notification services through an interactive and user-friendly platform.

Specific Objectives

1. To Provide Real-Time Flight Tracking

The system enables users to search for flights using flight numbers and view live flight information, including departure and arrival times, airline details, gate information, and current flight status.

2. To Display Accurate Flight Information

The project provides users with complete flight details such as departure and arrival airports, scheduled and estimated timings, terminal information, and flight status, ensuring that users receive reliable and up-to-date information.

3. To Integrate Live Flight Data

The system integrates REST APIs to retrieve real-time flight information from external aviation data providers and automatically updates the displayed information using polling techniques.

4. To Provide Flight Notifications

The application allows users to subscribe to selected flights and receive notifications regarding delays, cancellations, gate changes, boarding updates, and schedule modifications.

5. To Develop a User-Friendly Interface

The project aims to create a simple, interactive, and responsive interface that can be easily used by passengers, airline staff, and other users on desktops, tablets, and mobile devices.

6. To Support Flight Management

- 1.1** The system includes an administrative panel that enables administrators to manage flight records, monitor user activities, maintain historical flight data, and oversee notification services.

7. To Reduce Dependence on Manual Flight Enquiries

The project minimizes the need to rely on airport display boards, airline service counters, and repeated website refreshes by providing real-time flight information through a centralized web application.

8. To Improve the Passenger Experience

The system helps passengers stay informed about flight schedules and operational changes, enabling them to plan their journeys more efficiently and reducing confusion caused by unexpected flight updates.

9. To Provide a Centralized Flight Monitoring Platform

The application allows users to monitor multiple flights through a single dashboard, making it easier to access flight information from different airlines in one place.

10. To Establish a Foundation for Future Enhancements

The project is developed using scalable technologies, allowing future integration of advanced features such as AI-based delay prediction, weather analysis, mobile applications, cloud storage, SMS and email notifications, and airport navigation services.

Summary of Objectives

The main objectives of the Flight Status Tracker are:

- To provide real-time flight tracking.
- To display accurate and updated flight information.
- To integrate REST APIs for live flight data.
- To provide flight notifications and alerts.
- To develop an interactive and responsive web application.

- To support administrative management.
- To reduce dependence on traditional flight enquiry methods.
- To improve the overall passenger experience.
- To create a scalable platform for future enhancements.

The successful achievement of these objectives will provide passengers and airline personnel with a reliable, efficient, and user-friendly flight monitoring system that improves access to real-time flight information and enhances the overall travel experience.

1.5 Importance of the Application

The Flight Status Tracker plays a significant role in providing passengers, airline personnel, airport authorities, and travel agencies with accurate and real-time flight information through a centralized web-based platform. As the number of air travelers continues to increase worldwide, accessing reliable flight information has become essential for efficient travel planning and airport operations. The application addresses the challenges associated with obtaining timely flight updates by providing users with instant access to flight schedules, departure and arrival times, gate information, and current flight status. The importance of the application lies in its ability to simplify flight monitoring. Passengers often face inconvenience due to unexpected flight delays, gate changes, cancellations, or schedule modifications. By integrating live flight data through REST APIs and automatically updating the information using polling techniques, the system ensures that users always have access to the latest flight information. This reduces uncertainty, saves time, and enables passengers to plan their journeys more effectively. Another important aspect of the application is its notification service. Users can subscribe to specific flights and receive alerts regarding delays, cancellations, boarding updates, gate changes, and other important events. These notifications help passengers, family members, and airport staff respond quickly to schedule changes, improving communication and reducing confusion. The application also promotes digital transformation in the aviation sector by replacing the need for manual enquiries, repeated website refreshes, and dependence on airport display boards with a modern and responsive web platform. Users can monitor flights anytime and from anywhere using a desktop, tablet, or mobile device, making flight information more accessible and convenient. Furthermore, the system improves operational efficiency by providing an administrative panel through which authorized users can manage flight records, user information, historical flight data, and notification services. This centralized management approach ensures that accurate and updated information is maintained throughout the system while supporting efficient monitoring and maintenance.

The Flight Status Tracker is important because it:

- Provides real-time flight status updates.
- Displays accurate departure, arrival, gate, and terminal information.
- Reduces uncertainty caused by delays and cancellations.
- Sends instant notifications for important flight updates.
- Eliminates dependence on manual flight enquiries and airport display boards.
- Improves convenience for passengers and airline personnel.
- Supports efficient flight monitoring through a centralized dashboard.
- Provides a responsive and user-friendly web interface.
- Demonstrates the effective use of REST APIs and real-time data synchronization.
- Offers a scalable platform for future enhancements and additional aviation services.

The project also demonstrates the practical implementation of modern web development technologies. By using React.js for the frontend, Node.js for the backend, and optional MySQL for data storage, the

application provides a scalable and responsive architecture capable of handling real-time flight information efficiently. The use of REST API integration and polling mechanisms ensures continuous synchronization of flight data while maintaining system performance.

The architecture of the system is designed to support future enhancements such as AI-based flight delay prediction, weather-based flight analysis, SMS and email notifications, mobile application support, cloud-based storage, and integration with additional airline and airport services. This modular approach makes the application flexible and suitable for future technological advancements.

From an educational perspective, the project demonstrates the practical application of software engineering concepts such as requirement analysis, frontend and backend development, REST API integration, database management, responsive web design, and real-time data processing. It provides valuable experience in developing a full-stack web application that solves a real-world problem using modern technologies.

In conclusion, the Flight Status Tracker is an important application that combines real-time flight tracking, live notifications, and centralized flight information management into a single platform. By improving accessibility, enhancing communication, and providing accurate flight updates, the application contributes to a better travel experience while demonstrating the effective use of modern web technologies in the aviation industry.

1.6 Target Users / Audience

The **Flight Status Tracker** is designed to serve a wide range of users involved in air travel and airport operations. The application provides real-time flight information, live status updates, notifications, and flight monitoring services through a centralized web platform. The target audience includes passengers, airline staff, airport authorities, travel agencies, and system administrators who require accurate and timely flight information.

1. Passengers

Passengers are the primary users of the system. They can search for flights using flight numbers and view important details such as departure and arrival times, gate information, terminal details, and current flight status. The application also allows passengers to receive notifications about delays, cancellations, boarding updates, and schedule changes, helping them plan their journey more efficiently.

2. Airline Staff

Airline employees can use the application to monitor flight schedules and operational updates. The system helps staff stay informed about changes in flight timings, gate assignments, and flight status, enabling them to provide accurate information and better assistance to passengers.

3. Airport Authorities

Airport authorities can utilize the Flight Status Tracker to monitor multiple flights simultaneously through the flight status dashboard. The application supports efficient management of flight operations by providing real-time information and helping coordinate airport activities.

4. Travel Agencies

Travel agencies can use the application to track the flights of their clients and provide updated travel

information. This enables travel agents to quickly inform customers about delays, cancellations, or schedule modifications and improve customer service.

5. Family Members and Friends

Family members and friends waiting to receive passengers can monitor flight arrivals and departures in real time. This helps them plan airport pickups more effectively and reduces unnecessary waiting caused by unexpected flight delays.

6. System Administrators

Administrators are responsible for managing user accounts, monitoring application performance, maintaining historical flight data, and managing notification services. The admin panel enables efficient system management and ensures that users receive accurate and up-to-date flight information.

Benefits for the Target Audience

The application provides several benefits to its users:

- Easy access to real-time flight information.
- Instant notifications for delays, cancellations, and gate changes.
- Reduced dependence on airport display boards and manual enquiries.
- Improved travel planning and decision-making.
- Centralized monitoring of multiple flights.
- Accurate and reliable flight status updates.
- User-friendly and responsive web interface.
- Better communication between passengers, airlines, and airport authorities.

In conclusion, the **Flight Status Tracker** is designed to meet the needs of passengers, airline staff, airport authorities, travel agencies, family members, and administrators by providing a centralized platform for real-time flight monitoring and information management. The application improves accessibility, enhances communication, and delivers a convenient and reliable flight tracking experience for all users.

CHAPTER -2 SYSTEM REQUIREMENTS

2.1 Hardware Requirements

Hardware requirements define the physical components needed for the development, testing, and execution of the Flight Status Tracker application. Since the proposed system is a web-based application developed using modern frontend and backend technologies, it does not require highly specialized hardware. However, a standard computer system with sufficient processing power, memory, and storage is necessary to ensure smooth development, real-time data processing, and efficient application performance.

The hardware configuration used for the Flight Status Tracker is adequate to support React.js development, Node.js server execution, REST API integration, real-time flight data updates, and responsive user interface rendering. Proper hardware resources help improve system performance, reduce lag during data fetching, and provide a better user experience.

The minimum hardware requirements for the proposed system are described below.

Processor

The processor is responsible for executing program instructions and handling multiple application tasks such as API requests, data processing, and UI rendering.

Minimum Requirement:

- Intel Core i3 Processor or Equivalent

Recommended Requirement:

- Intel Core i5 Processor or Higher

A higher processor speed ensures smooth execution of development tools, browsers, and backend services.

Memory (RAM)

RAM is required for running the operating system, development environment, web browser, and server processes simultaneously.

Minimum Requirement:

- 4 GB RAM

Recommended Requirement:

- 8 GB RAM or Above

Adequate memory improves system responsiveness and supports efficient multitasking during development and testing.

Storage

Storage is required for installing software tools, project files, dependencies, and system libraries.

Minimum Requirement:

- 20 GB Free Disk Space

Recommended Requirement:

- 50 GB Free Disk Space

Solid State Drives (SSD) are preferred for faster booting, quicker file access, and improved performance.

Display

A suitable display resolution is required for effective development, debugging, and visualization of dashboards and flight information.

Minimum Resolution:

- 1366 × 768 Pixels

Recommended Resolution:

- 1920 × 1080 Pixels

Higher resolution displays improve clarity and enhance user interface design and testing.

Input Devices

The following input devices are required for system operation and development:

- Keyboard
 - Mouse or Touchpad
- These devices are essential for coding, navigation, and system interaction.
- Internet Connectivity**
- The Flight Status Tracker relies on real-time flight data fetched through REST APIs; therefore, a stable internet connection is required.
- Requirement:**
- **Broadband Internet Connection**
- A reliable internet connection ensures uninterrupted access to live flight data and smooth API communication.
- Hardware Configuration Table**
- | Hardware Component | Specification |
|--------------------|-------------------------|
| Processor | Intel Core i3 or Higher |
| RAM | 4 GB Minimum |
| Storage | 20 GB Free Space |
| Display | 1366 × 768 Resolution |
| Input Devices | Keyboard and Mouse |
| Internet | Broadband Connection |
- The above hardware configuration is sufficient for the successful development, testing, and deployment of the Flight Status Tracker. It ensures stable performance for real-time flight tracking, API integration, dashboard rendering, and responsive web application usage.

2.2 Software Requirements

Software requirements refer to the collection of software tools, programming languages, frameworks, libraries, operating systems, and development environments required for the successful development, testing, and execution of the **Flight Status Tracker** application. The selection of appropriate software technologies is essential to ensure system reliability, performance, scalability, and maintainability.

The Flight Status Tracker is developed as a web-based application using modern full-stack technologies that support real-time flight data processing, interactive dashboards, API integration, and notification services. These software components provide the necessary environment for implementing and managing all system functionalities.

Operating System

The application can be developed and executed on multiple operating systems, including:

- Windows 10 / Windows 11
- Linux
- macOS

These operating systems provide a stable platform for running development tools, browsers, and server environments.

Programming Languages

JavaScript

JavaScript is the primary programming language used in the system. It is used for implementing both frontend and backend logic, handling user interactions, API requests, and real-time updates.

HTML5

HTML5 is used to structure the web pages and define the layout of the application, ensuring a clean and organized interface.

CSS3

CSS3 is used for styling the application and creating a responsive, visually appealing, and user-friendly design.

Frontend Framework

React.js

React.js is used for building the interactive user interface of the Flight Status Tracker. Its component-based architecture improves performance, reusability, and maintainability of the application.

Backend Environment

Node.js

Node.js is used as the runtime environment for executing server-side JavaScript. It handles API requests, data processing, and communication with external flight data services.

Express.js (Optional Framework)

Express.js can be used to simplify backend development by providing routing, middleware support, and efficient API handling.

Build Tool

Vite

Vite is used as a modern frontend build tool that provides fast development server startup, hot module replacement, and optimized production builds.

Development Environment

Visual Studio Code

Visual Studio Code (VS Code) is used as the primary code editor. It provides features such as debugging support, extensions, syntax highlighting, and integrated terminal support.

Database (Optional)

MySQL

MySQL is used for storing user data, notification preferences, flight history, and system logs. It ensures structured and efficient data management.

APIs and Data Integration

REST APIs

REST APIs are used to fetch real-time flight data from external aviation service providers.

JSON Format

JSON is used for data exchange between frontend, backend, and external APIs.

Browser Requirements

The application can be accessed using modern web browsers such as:

- Google Chrome
- Mozilla Firefox
- Microsoft Edge
- Safari

These browsers support modern JavaScript features and ensure smooth application performance.

Additional Software Tools

npm (Node Package Manager)

npm is used to install and manage project dependencies and libraries.

Git

Git is used for version control, tracking changes, and collaborative development.

Postman

Postman is used for testing REST APIs and verifying backend responses during development.

Software Configuration Table

Software Component	Purpose
Windows/Linux/macOS	Operating System
JavaScript	Application Logic
HTML5	Page Structure
CSS3	Styling
React.js	Frontend Development
Node.js	Backend Runtime
Express.js	API Handling
Vite	Build Tool
MySQL	Database Management

Visual Studio Code	Code Editor
npm	Package Management
Git	Version Control
Postman	API Testing
REST APIs	Flight Data Integration

Advantages of the Selected Software

The selected software technologies provide several advantages:

- Fast and efficient development of web applications
- Real-time flight data integration using REST APIs
- Responsive and interactive user interface
- Scalable and modular architecture
- Easy maintenance and debugging
- Cross-platform compatibility
- Improved performance and user experience
- Support for future enhancements such as AI-based predictions and mobile applications

In conclusion, the combination of these software technologies provides a robust and scalable environment for developing the **Flight Status Tracker**, ensuring reliable performance, real-time updates, and an enhanced user experience for monitoring flight information.

2.3 Development Tools and Frameworks

Development tools and frameworks play a crucial role in the successful design, implementation, testing, and maintenance of the **Flight Status Tracker** application. These tools provide an efficient development environment, simplify coding tasks, improve productivity, and ensure the delivery of a high-quality, scalable, and responsive web application. The project utilizes modern full-stack technologies to create an interactive platform for real-time flight tracking, data visualization, and notification management.

The selected development tools and frameworks support component-based development, real-time data handling, API integration, efficient debugging, and optimized application performance. They enable developers to build scalable applications while maintaining code quality, consistency, and maintainability.

React.js

React.js is the primary frontend library used for developing the user interface of the Flight Status Tracker. It is an open-source JavaScript library developed by Meta for building dynamic and interactive web applications. React follows a component-based architecture that allows developers to create reusable UI components and manage application state efficiently.

React.js offers several advantages, including fast rendering through the Virtual DOM, reusable components, improved performance, and simplified maintenance. In the Flight Status Tracker, React.js is used to develop modules such as flight search, flight dashboard, notification system, airline filters, and user interface components.

Features of React.js:

- Component-based architecture

- Virtual DOM for faster rendering
 - Reusable UI components
 - Efficient state management
 - High performance and scalability
 - Easy maintenance and debugging
-

Vite

Vite is a modern frontend build tool used for developing and managing the Flight Status Tracker application. It provides a fast development environment with instant server startup and efficient hot module replacement.

Compared to traditional build tools, Vite significantly improves development speed and productivity by allowing instant updates without rebuilding the entire application.

Features of Vite:

- Fast development server startup
 - Instant Hot Module Replacement (HMR)
 - Optimized production builds
 - Lightweight configuration
 - Improved development efficiency
-

Visual Studio Code

Visual Studio Code (VS Code) is the integrated development environment used for writing and managing the project code. It provides powerful features such as syntax highlighting, debugging support, intelligent code completion, and extension integration.

VS Code helps developers write clean, organized, and efficient code while improving overall productivity.

Features of Visual Studio Code:

- Syntax highlighting and IntelliSense
 - Code auto-completion
 - Debugging support
 - Integrated terminal
 - Extension marketplace
-

- Git integration
-

Node.js

Node.js is used as the backend runtime environment for executing JavaScript outside the browser. It plays a key role in handling server-side logic, API requests, and real-time flight data processing.

Node.js enables efficient handling of multiple concurrent requests, making it suitable for real-time applications like flight tracking systems.

Features of Node.js:

- Event-driven architecture
 - Fast and scalable runtime environment
 - Non-blocking I/O operations
 - Cross-platform support
 - Efficient handling of API requests
-

npm (Node Package Manager)

npm is the package manager used in the Flight Status Tracker project. It is used to install, update, and manage external libraries and dependencies required by the application.

The project uses npm to manage packages such as React.js, Axios, React Router, and other supporting libraries for API communication and UI development.

Features of npm:

- Dependency management
 - Easy package installation
 - Version control support
 - Access to open-source libraries
 - Simplified project maintenance
-

REST APIs

REST APIs are used to fetch real-time flight data from external aviation service providers. They enable communication between the frontend, backend, and external data sources.

In the Flight Status Tracker, REST APIs provide live flight information such as departure times, arrival times, gate details, and flight status updates.

Features of REST APIs:

- Real-time data communication
 - Lightweight and scalable architecture
 - JSON-based data exchange
 - Platform-independent communication
 - Easy integration with web applications
-

MySQL (Optional Database)

MySQL is used for storing user data, notification preferences, and flight history. It provides structured data storage and ensures data consistency and reliability.

Features of MySQL:

- Secure and structured data storage
 - High performance and reliability
 - Support for complex queries
 - Scalable database management
 - Data integrity and consistency
-

Development Tools and Frameworks Summary

Tool / Framework	Purpose
React.js	Frontend User Interface Development
Vite	Build Tool and Development Server
Node.js	Backend Runtime Environment
npm	Package Management
Visual Studio Code	Code Editor
REST APIs	Real-time Flight Data Integration

The combination of these development tools and frameworks provides a powerful and efficient environment for building the **Flight Status Tracker**. They ensure high performance, real-time data handling, scalability, and a smooth user experience while supporting future enhancements such as AI-based predictions, mobile applications, and advanced notification systems.

.

CHAPTER-3 TECHNOLOGY STACK

3.1 Front-End Design

The frontend design of the **Flight Status Tracker** focuses on creating an interactive, responsive, and user-friendly interface that allows users to access real-time flight information efficiently. The frontend acts as the visual and interactive layer of the application, enabling users to search flights, view live status updates, check flight details, and receive notifications through a well-structured and intuitive design.

The Flight Status Tracker is developed using **React.js**, a modern JavaScript library that supports component-based development. This approach allows the application to be divided into reusable modules such as flight search, flight dashboard, flight details view, airline filters, notification system, and administrative interface. The modular structure improves code reusability, maintainability, and scalability of the application.

The frontend is designed to ensure that users can easily search for flights, track real-time status, view departure and arrival details, and receive important updates without requiring technical knowledge. A simple and clean user interface is implemented to enhance usability for passengers, airline staff, travel agents, and general users.

One of the key features of the frontend design is **responsiveness**. The application is optimized to work smoothly across different devices, including desktop computers, laptops, tablets, and smartphones. Responsive layouts ensure that users receive a consistent and seamless experience regardless of screen size or device type.

The frontend interface consists of multiple components that work together to provide the overall functionality of the system. These components include:

Home Page Interface

The home page serves as the entry point of the application. It provides an overview of the Flight Status Tracker and allows users to access key features such as flight search, live tracking, and notifications.

Flight Search Interface

The flight search interface allows users to enter a flight number and retrieve real-time flight information. It is designed for quick and efficient access to flight details.

Flight Dashboard Interface

The flight dashboard displays multiple flights simultaneously, enabling users to monitor different flight statuses in a single view. It improves usability for frequent travelers and airline personnel.

Flight Details Interface

This interface displays complete flight information such as airline name, departure and arrival airports, scheduled and estimated timings, gate details, terminal information, and current flight status.

Notification Interface

The notification interface displays alerts related to flight delays, cancellations, boarding updates, gate changes, and schedule modifications. It ensures users stay informed about important updates.

Filtering Interface

This component allows users to filter flights based on airline, flight status, or schedule, making it easier to find specific flight information quickly.

Administrative Interface

The administrative interface allows authorized users to manage flight data, monitor system activity, handle notifications, and maintain historical flight records.

Frontend Design Principles

The frontend design follows several important principles:

- Simplicity and ease of use
 - Responsive user interface design
 - Consistent layout and navigation structure
 - Fast and efficient data rendering
 - Interactive and real-time updates
 - Reusable component-based architecture
 - User-friendly interaction mechanisms
 - Clear and structured information display
-

Advantages of the Frontend Design

The frontend design provides several benefits:

- Improved user experience and accessibility
 - Easy navigation and quick access to flight data
 - Clear and structured presentation of information
 - Responsive performance across all devices
 - Efficient real-time interaction with flight updates
 - Scalable component-based architecture
 - Better maintainability and code organization
 - Smooth integration with backend APIs
-

In conclusion, the frontend design of the **Flight Status Tracker** provides a modern, responsive, and interactive user interface that supports efficient flight tracking and real-time information access. The use of **React.js** and modern web technologies ensures a high-quality user experience while maintaining scalability and flexibility for future enhancements such as advanced analytics, mobile applications, and AI-based predictions.

3.2 Back-End Design

The **Flight Status Tracker** uses a backend system to handle data processing, API requests, and communication between the frontend and external flight data sources. The backend is developed using **Node.js** with **Express.js**, which provides a simple and efficient framework for creating RESTful APIs.

The backend is responsible for fetching real-time flight data from external **REST APIs**, processing the information, and sending it to the frontend in JSON format. It ensures that users receive updated flight details such as departure time, arrival time, gate information, and flight status.

The system follows a REST-based architecture where the backend acts as a bridge between the frontend and external aviation services. It also supports polling to update flight information at regular intervals for real-time tracking.

Optionally, a **MySQL database** can be used to store user preferences, notification data, and flight history for future enhancements.

Advantages

- Simple and efficient server-side processing
- Easy integration with external APIs
- Supports real-time flight updates
- Scalable and modular design

Limitations

- Limited advanced backend features in basic version
- Depends on external APIs for data
- Optional database not fully utilized

Future Enhancements

Future versions can include authentication systems, cloud database integration, real-time WebSocket updates, and notification services such as email or SMS alerts.

In conclusion, the backend design provides a lightweight and efficient system for handling real-time flight data and ensures smooth communication between the frontend and external services.

3.3 Database

Database design refers to the process of organizing and managing data in a structured format using a Database Management System (DBMS). In many web applications, databases are used to store user details, application data, transaction history, and other persistent information required for system functionality.

In the current implementation of the **Flight Status Tracker**, a dedicated database is used in an optional and limited manner. The main focus of the project is on real-time flight tracking through external **REST APIs**, where live flight data is fetched dynamically and displayed to the user.

Since most flight information is obtained directly from external aviation data providers, permanent storage of flight details is not required in the core system. However, a database such as **MySQL** can be optionally used to store user preferences, notification subscriptions, and historical flight search records.

The application primarily relies on API-based data flow rather than structured database tables, relationships, or complex schemas. This reduces system complexity and ensures faster access to real-time flight information.

Reasons for Limited Database Usage

- The system focuses mainly on real-time API integration.
 - Flight data is dynamically fetched from external sources.
 - No heavy transactional data is required.
 - Reduces system complexity and storage overhead.
 - Faster performance and easier implementation.
-

Future Scope of Database Integration

In future versions of the Flight Status Tracker, a database can be integrated to support advanced functionalities such as:

- User registration and login system
- Saving favorite or frequently tracked flights
- Storing notification history
- Flight search history tracking
- Analytics and reporting features

Possible database technologies include:

- MySQL
 - PostgreSQL
 - MongoDB
 - Firebase Firestore
 - SQLite
-

In conclusion, the current version of the **Flight Status Tracker** relies primarily on real-time API-based data rather than a full-scale database system. However, database integration is identified as a key enhancement for future development to support advanced features and improved user personalization.

3.4 Version Control

Version control is an important aspect of software development that helps developers manage changes made to source code, track project progress, and maintain different versions of the application. It ensures better collaboration, reduces the risk of code loss, and provides a structured approach to software development and maintenance.

In the development of the **Flight Status Tracker**, version control is used to organize project files, monitor code changes, and maintain consistency throughout the development process. It helps developers track updates made to frontend components, backend APIs, and configuration files, ensuring smooth development and debugging.

The most commonly used version control system for this project is **Git**. Git is a distributed version control system that allows developers to record changes, manage project history, and work on different features simultaneously using branches.

Version control provides several advantages during development. It enables developers to maintain backups of source code, track modifications, and restore previous versions when needed. It also supports collaborative development by allowing multiple developers to work on different modules such as flight search, API integration, and notification systems.

Features of Version Control

- Tracks changes made to source code
-

- Maintains project history and version records
 - Provides backup and recovery options
 - Supports collaborative development
 - Helps identify and fix errors easily
 - Enables restoration of previous versions
 - Improves project organization and maintenance
-

Git as a Version Control Tool

Git is widely used in modern web development due to its speed, reliability, and flexibility. It allows developers to create repositories, manage code changes, and work on multiple branches for different features such as flight dashboard, API integration, and notification modules.

Benefits in Flight Status Tracker

For the **Flight Status Tracker**, version control helps in:

- Managing React.js frontend files efficiently
 - Tracking updates in API integration and backend services
 - Maintaining different versions of UI components
 - Organizing project structure and configurations
 - Supporting feature upgrades and enhancements
 - Ensuring code stability during development
-

Summary

Version control plays a crucial role in the successful development of the **Flight Status Tracker**. By using Git, developers can efficiently manage source code, track changes, collaborate effectively, and maintain a stable and scalable development workflow. This ensures better organization, reliability, and long-term maintainability of the application.

3.5 APIs or External services

The **Flight Status Tracker** utilizes several external services and APIs to provide real-time flight information, data visualization, and live status updates. These services play a crucial role in ensuring that the application delivers accurate and up-to-date flight details without requiring a fully dedicated backend data infrastructure.

The system primarily depends on aviation-related **REST APIs** and browser-based services to fetch live flight data, process it, and display it in a user-friendly format. These external integrations enhance the functionality of the application and improve the overall user experience.

Geolocation API

The Geolocation API is a browser-based service used to obtain the current location of the user. In the Flight Status Tracker, it can be used to provide location-based flight suggestions such as nearby airports or regional flight information.

Features:

- Retrieves user's current geographical location
 - Supports location-based flight search enhancements
 - Improves personalization of flight tracking
 - Works directly in modern web browsers
-

Flight Data REST APIs

The Flight Data REST APIs are the core external services used in the system. These APIs provide real-time flight information such as departure times, arrival times, gate details, airline information, and current flight status. These APIs act as the primary data source for the application.

Features:

- Provides real-time flight status updates
 - Supplies departure and arrival information
 - Includes gate, terminal, and delay details
 - Returns structured data in JSON format
 - Supports continuous data updates
-

Airline/Aviation Data Providers

The system integrates with external aviation data providers that supply global flight tracking information. These services ensure that the application always displays accurate and live flight data.

Features:

- Global flight coverage
 - Real-time tracking updates
 - Accurate airline schedule information
 - Reliable aviation data sources
-

Browser APIs

Modern browser APIs are also used to support application functionality and improve user experience.

Features:

- Local storage for saving user preferences (optional)
 - Fetch API for handling HTTP requests
 - Real-time data rendering in the frontend
 - Smooth interaction with external services
-

External Services Summary

Service / API	Purpose
---------------	---------

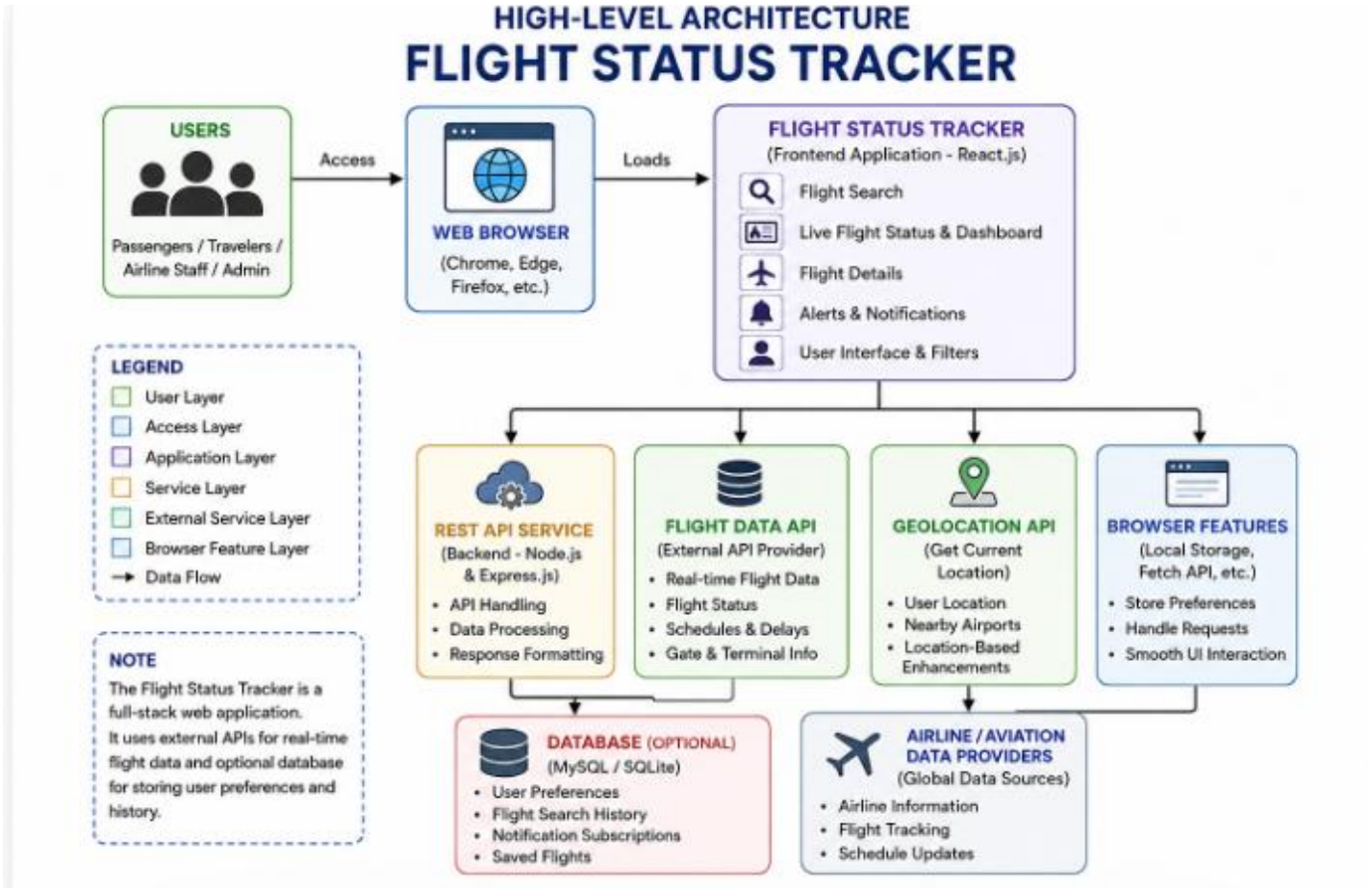
Geolocation API	User location detection
Flight REST APIs	Live flight data retrieval
Aviation Data Providers	Airline and schedule information
Browser APIs	Data handling and UI interaction

Conclusion

The **Flight Status Tracker** does not rely heavily on traditional backend services such as payment gateways or complex server-side systems. Instead, it uses **Geolocation API, REST-based flight data services, aviation providers, and browser APIs** to deliver real-time flight tracking functionality. These integrations ensure accurate, fast, and reliable flight information while maintaining a lightweight and efficient system architecture.

CHAPTER -4 SYSTEM ARCHITECTURE

4.1 High-level architecture diagram



4.2 Description of Each Layer

1. User Layer

The User Layer represents the end users who interact with the Flight Status Tracker application. The primary users include passengers, airline staff, travel agencies, airport authorities, and administrators. Users access the system to search flights, view real-time flight status, check arrival and departure details, and receive notifications.

Responsibilities:

- Access the application
- Search flight information using flight number
- View live flight status and updates
- Receive notifications for delays or cancellations
- Interact with dashboards and filters

2. Access Layer (Web Browser)

The Access Layer consists of web browsers such as Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari. This layer acts as the interface between the user and the application.

Responsibilities:

- Load the Flight Status Tracker application

- Render frontend UI components
 - Handle user requests
 - Support browser APIs (Fetch, Local Storage)
 - Enable smooth user interaction
-

3. Application Layer (React.js Frontend)

The Application Layer is the core frontend system developed using React.js. It contains all major modules responsible for user interaction and data display.

Responsibilities:

- Manage UI components (Flight Search, Dashboard, Filters)
- Handle user inputs and interactions
- Display real-time flight information
- Process API responses
- Manage notifications and alerts

Main Modules:

- Flight Search Module
 - Flight Dashboard Module
 - Flight Details Module
 - Notification Module
 - Filter & Sorting Module
 - Admin Interface
-

4. Service Layer (Backend – Node.js & Express.js)

The Service Layer acts as the backend of the system. It is developed using Node.js and Express.js and handles API requests and data processing.

Responsibilities:

- Handle REST API requests
 - Fetch flight data from external APIs
 - Process and format data into JSON
 - Manage polling for real-time updates
 - Ensure communication between frontend and external services
-

5. External Flight Data API Layer

This layer consists of external aviation and flight tracking APIs that provide real-time flight information.

Responsibilities:

- Provide live flight status updates
 - Supply departure and arrival details
 - Provide gate, terminal, and delay information
 - Deliver structured JSON responses
 - Ensure real-time data accuracy
-

6. Browser Features Layer

This layer includes built-in browser services used by the application.

Responsibilities:

- Handle API requests (Fetch/Axios)
 - Store temporary user data (Local Storage)
 - Manage session-based interactions
 - Support real-time UI updates
 - Improve application performance
-

7. Database Layer (Optional)

A database is optionally used in the system for storing user-related and historical data.

Responsibilities:

- Store user preferences
 - Save notification subscriptions
 - Maintain flight search history
 - Store admin records (optional)
-

8. Airline / Aviation Data Providers

This layer represents global aviation data providers that supply real-time and historical flight data.

Responsibilities:

- Provide global flight tracking data
 - Offer airline schedule information
 - Supply real-time updates and delays
 - Ensure accurate aviation records
-

Working of the Architecture

The user accesses the Flight Status Tracker through a web browser, which loads the React.js application. When a flight is searched, the frontend sends a request to the backend service (Node.js + Express.js). The backend fetches real-time flight data from external aviation APIs, processes it, and returns it to the frontend in JSON format. The browser then displays the updated flight information in real time. Optional database support can store user preferences and flight history for future use.

Figure Description

The architecture illustrates a layered system where users interact through a web browser with a React.js frontend application. The frontend communicates with a Node.js backend service, which connects to external flight data APIs. Browser features support data handling, while optional databases store user-related information. This layered structure ensures real-time flight tracking, efficient data flow, and a responsive user experience.

4.3 Deployment Architecture

The deployment architecture of the **Flight Status Tracker System** describes how the application is built, executed, and made accessible to users. The current implementation follows a **frontend-based deployment model**, where the application runs entirely in a web browser without requiring a dedicated backend server or database infrastructure.

The system is developed using **ReactJS and Vite**, enabling a fast and responsive single-page application. Users can access the system through modern web browsers such as Google Chrome, Mozilla Firefox, Microsoft Edge, or Safari. The application communicates with external aviation data services and APIs to retrieve real-time flight information.

In the current version, most processing is handled on the **client side**. Flight search, status display, and UI rendering are performed within the React application. Real-time flight data is obtained from third-party aviation APIs (such as flight tracking services), and the results are displayed dynamically to the user. Since the system does not maintain a dedicated backend, there is no requirement for application servers or database systems.

Local Deployment

The project is primarily deployed and tested in a **local development environment**.

Deployment Steps

- Install Node.js and npm
- Download the project source code
- Install project dependencies using npm
- Start the Vite development server
- Access the application through a web browser

This approach allows developers to quickly test and debug the system without requiring external infrastructure.

Cloud Deployment

The current implementation does not utilize cloud computing platforms such as:

- Amazon Web Services (AWS)
- Microsoft Azure
- Google Cloud Platform (GCP)
- Firebase Hosting

However, the system can be easily deployed to cloud platforms in future versions to enable global accessibility and improved scalability.

CI/CD Pipeline

Continuous Integration and Continuous Deployment (CI/CD) pipelines have not been implemented in the current version of the project. All development, testing, and deployment activities are performed manually. Future enhancements may include integration with CI/CD tools such as:

- GitHub Actions
- Jenkins
- GitLab CI/CD
- Azure DevOps

These tools can automate building, testing, and deployment processes, improving development efficiency and reliability.

Deployment Architecture Components

Component	Description
User Device	Accesses the Flight Status Tracker application
Web Browser	Executes the frontend application
ReactJS Application	Provides flight search and status tracking interface
Flight Data API	Supplies real-time flight status and schedule information
External Aviation Services	Provides live flight tracking and airline data

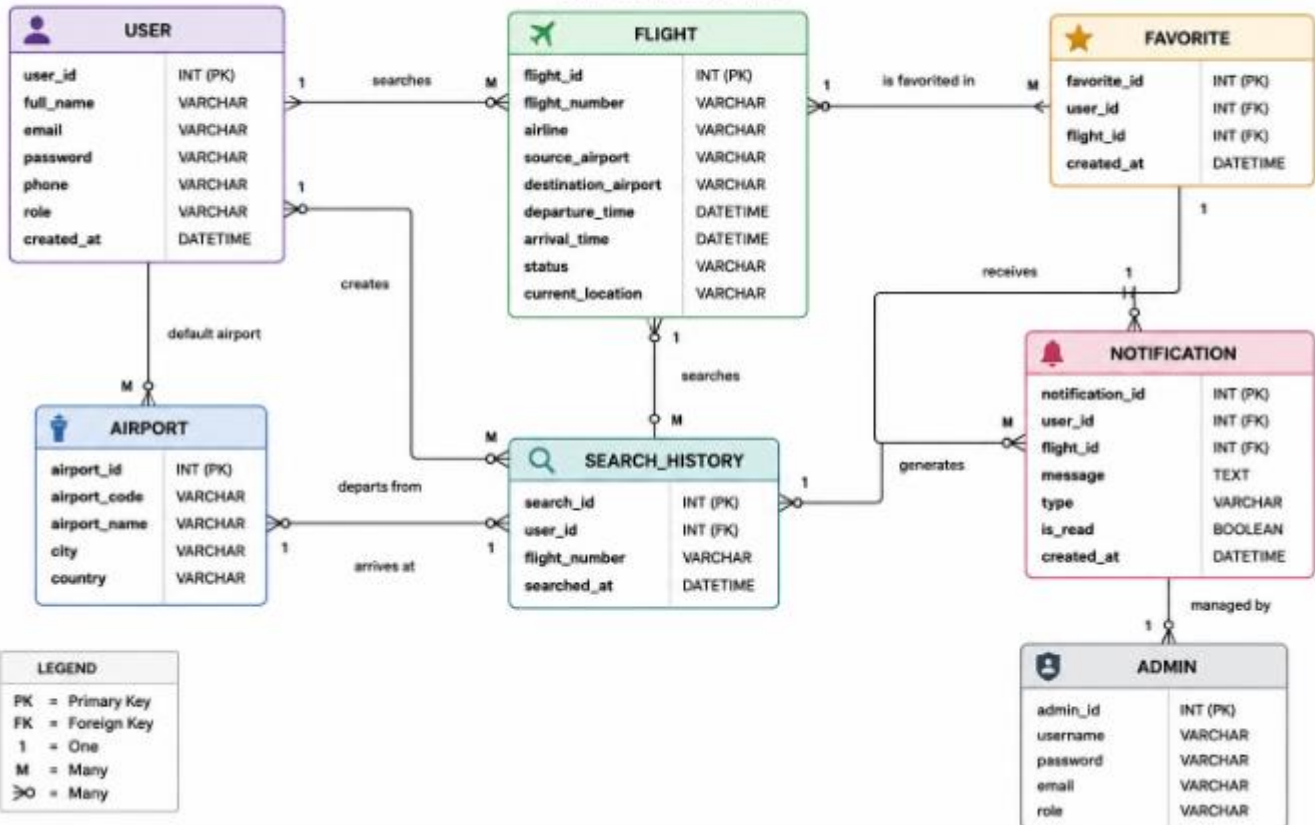
Advantages of Current Deployment Architecture

- Simple and lightweight architecture
- Easy deployment and maintenance
- No server management required
- Reduced infrastructure cost
- Fast development and testing cycle
- Real-time data integration through APIs

The **Flight Status Tracker System** currently follows a **client-side frontend deployment architecture**. The application operates entirely within the browser using ReactJS and external flight data APIs. No backend server, database, or cloud infrastructure is implemented in the current version. This architecture provides a simple, efficient, and scalable foundation that can be extended in future versions with backend services.

CHAPTER -5 DESIGN

FLIGHT STATUS TRACKER SYSTEM ER DIAGRAM



1.1 – Module-Wise Implementation

The Flight Status Tracker System is developed using a modular architecture in which the application is divided into multiple functional modules. Each module performs a specific task and collectively contributes to the overall functionality of the system. This approach improves maintainability, scalability, code reusability, and development efficiency.

The major modules implemented in the system are described below.

Authentication Module

The Authentication Module is responsible for controlling user access to the application. It ensures that only authorized users can access the system and its features.

Functions:

- User login and authentication.
- Credential validation.
- Session management.
- Access control for protected routes.

Components Used:

- AuthForm.jsx

Benefits:

- Improved system security.
- Controlled access to flight data.
- Secure user session handling.

Flight Search Module

The Flight Search Module allows users to search for flights using flight number, airline name, or route details.

Functions:

- Flight number search.
- Airline-based search.
- Source and destination filtering.
- Auto-suggestions for flight queries.

Components Used:

- FlightSearch.jsx

Benefits:

- Quick flight lookup.
- Easy navigation of flight information.
- Improved user convenience.

Flight Status Tracking Module

This is the core module of the system that provides real-time flight status information.

Functions:

- Live flight status updates (on-time, delayed, cancelled, landed).
- Departure and arrival tracking.
- Gate and terminal information.
- Estimated time updates.

Components Used:

- FlightStatusCard.jsx

Benefits:

- Accurate real-time flight tracking.
- Reduced uncertainty for passengers.
- Improved travel planning.

Live Flight Update Module

This module continuously fetches and updates flight data in real time.

Functions:

- Periodic API data fetching.
- Auto-refresh of flight status.
- Live delay and schedule updates.

Components Used:

- LiveUpdateService.jsx

Technologies Used:

- Flight Data API
- JavaScript Fetch/Axios

Benefits:

- Real-time accuracy.
- Reduced manual refresh requirement.
- Up-to-date flight information.

Flight Details Module

The Flight Details Module provides complete information about selected flights.

Functions:

- Display flight route details.
- Aircraft information.
- Departure/arrival timing.
- Duration and layover details.

Components Used:

- FlightDetails.jsx

Benefits:

- Comprehensive flight information.
- Better user understanding.
- Improved decision-making.

Map Visualization Module

This module visually represents flight routes and airport locations on an interactive map.

Functions:

- Flight route mapping.
- Airport location display.
- Aircraft movement visualization (if enabled).

Components Used:

- MapView.jsx

Technologies Used:

- Leaflet Maps
- OpenStreetMap

Benefits:

- Better route visualization.
- Interactive user experience.
- Easier spatial understanding of flights.

Notification Module

The Notification Module alerts users about important flight updates.

Functions:

- Delay notifications.
- Gate change alerts.
- Cancellation alerts.
- Status update messages.

Components Used:

- Notifications.jsx

Benefits:

- Timely updates.
 - Improved passenger awareness.
 - Reduced chances of missing flights.
-

Flight History Module

This module stores and displays previously searched or tracked flights.

Functions:

- Search history tracking.
- Recently viewed flights.
- Saved flights list.

Components Used:

- FlightHistory.jsx

Benefits:

- Easy access to past searches.
 - Improved user experience.
 - Faster repeated tracking.
-

Help and Support Module

The Help and Support Module provides guidance for using the application effectively.

Functions:

- User assistance.
- FAQs.
- Usage instructions.

Components Used:

- HelpSupport.jsx

Benefits:

- Improved usability.
 - Better user understanding.
 - Reduced user confusion.
-

Administrative Module

The Administrative Module allows system administrators to manage flight data and monitor system activity.

Functions:

- Flight data management.
- System monitoring.
- User activity tracking.
- Data updates and control.

Components Used:

- AdminDashboard.jsx
- AdminLoginGate.jsx

Benefits:

- Efficient system control.
- Better data management.
- Improved platform reliability.

Conclusion

The Flight Status Tracker System is implemented using multiple ReactJS modules that collectively provide flight search, real-time tracking, notifications, map visualization, and administrative control. The modular architecture improves maintainability, supports future enhancements, and ensures an efficient and user-friendly experience for travelers.

1.2 – Front-End Logic

The **Flight Status Tracker System** is developed using **ReactJS**, which follows a component-based architecture to build a responsive and interactive user interface. The frontend is responsible for displaying all application screens, handling user interactions, managing application state, and dynamically updating flight information without requiring page reloads. The system provides a smooth and seamless experience for users while tracking flight details in real time across desktops, laptops, tablets, and mobile devices.

The user interface consists of multiple reusable React components, including the Home Page, Authentication Module, Flight Search Module, Flight Status Module, Flight Details Module, Live Update Module, Map Visualization Module, Notification Module, Flight History Module, Help and Support, and Admin Dashboard. Each component performs a specific function and communicates with other components to provide complete system functionality.

React uses the Virtual DOM to improve rendering performance. Whenever the user interacts with the application, such as searching for a flight, selecting a route, or viewing live status updates, only the affected components are re-rendered instead of refreshing the entire page. This results in faster performance and an improved user experience, especially when handling continuously updating flight data.

The application uses **React Hooks** for managing the state of different components. The `useState()` hook is used to store dynamic information such as flight number, airline details, selected flight, live status updates, delay information, and notification messages. Whenever these values change, the interface updates automatically to reflect the latest flight information.

The `useEffect()` hook is used to perform side effects such as fetching real-time flight data from APIs, updating live flight status, initializing map components, loading flight routes, and automatically refreshing flight information at regular intervals. It ensures that the system always displays up-to-date flight details.

Data is shared between different components using **Props and callback functions**. For example, when a user searches for a flight in the Flight Search Module, the selected flight data is passed to the Flight Status Module, which displays real-time status information. Similarly, the Map Visualization Module receives flight route details and displays the flight path between departure and arrival airports. The Notification Module receives updates and alerts based on changes in flight status.

The frontend is designed to be fully responsive, ensuring that all pages automatically adjust to different screen sizes and devices. CSS3 and React component styling are used to create a clean, modern, and user-friendly interface that enhances usability and accessibility for travelers checking flight information.

Advantages of the Frontend Logic

- Component-based architecture improves code reusability and maintainability.
- Virtual DOM enables faster rendering and efficient updates.
- Real-time state management improves user interaction and accuracy.

- Responsive design supports desktops, tablets, and mobile devices.
- Easy scalability for adding new flight tracking features.
- Enhanced user experience through interactive and dynamic interfaces.

1.3 API Integration

The **Flight Status Tracker System** uses external APIs and browser services to provide real-time flight information without a dedicated backend server.

The main data source is a **Flight Data API**, which provides live updates such as flight status, departure and arrival times, delays, and gate information. This data is fetched directly from the frontend and displayed to users in real time.

The system also uses the **Geolocation API** to get the user's location for enhanced search features. For visualization, **Leaflet Maps with OpenStreetMap (OSM)** is used to display airports, flight routes, and interactive maps.

When a user searches for a flight, the system sends a request to the API and dynamically updates the UI with the latest flight details.

APIs and Services Used

- **Flight Data API** – Provides real-time flight status updates
- **Geolocation API** – Gets user location (optional feature)
- **OpenStreetMap (OSM)** – Provides map data
- **Leaflet Maps** – Displays interactive maps and routes

Advantages

- Real-time flight updates
- No backend required
- Fast and lightweight system
- Interactive map visualization
- Easy integration and maintenance

Authentication and Authorization

The system includes a simple authentication mechanism for admin access. Users log in through an **Admin Login Module**, where credentials are verified on the frontend.

Regular users can view and track flights, while admin users have additional access to manage flight data and system updates.

Since there is no backend, authentication is handled on the frontend, which is suitable for demonstration purposes.

User Roles

User:

- Search and track flights
- View flight status and details
- Access maps and notifications

Admin:

- Login to dashboard

- Manage flight information
 - Monitor system activity
-

Features

- Simple login system
 - Role-based access
 - Protected admin dashboard
-

Future Enhancements

- JWT authentication
- Firebase authentication
- OAuth login (Google)
- Password encryption
- Multi-factor authentication

CHAPTER 7- FEATURES

7.1 List of Main Features

The **Flight Status Tracker System** provides a real-time, user-friendly solution for tracking flight information and improving travel awareness. The application integrates flight search, live status updates, interactive maps, notifications, and administrative control into a single web-based platform. The major features of the system are described below.

1. User Authentication

The application provides a secure login system for administrators. Authentication ensures that only authorized users can access the admin dashboard, while general users can freely search and track flights.

2. Flight Search

Users can search for flights using flight number, airline name, or route information. This feature helps users quickly access relevant flight details without manually browsing through data.

3. Flight Status Display

The system provides real-time flight status updates such as on-time, delayed, cancelled, boarding, or landed. It ensures users receive accurate and up-to-date flight information.

4. Flight Details View

Users can view complete flight information including departure time, arrival time, terminal, gate details, and flight duration, helping them understand all essential travel details.

5. Interactive Map Visualization

The application uses Leaflet Maps and OpenStreetMap to display interactive flight routes and airport locations. Users can zoom, pan, and view flight paths visually on the map.

6. Live Flight Tracking

The system updates flight information in real time using external APIs. Users can track flight progress and status changes dynamically without refreshing the page.

7. Notification System

The Notification Module provides alerts for delays, gate changes, cancellations, and important flight updates to keep users informed in real time.

8. Flight History

The system stores previously searched flights, allowing users to quickly access their recent searches and track flights more efficiently.

9. Help and Support

The Help and Support section provides guidance on how to use the application, along with FAQs and user assistance to improve usability.

10. Administrative Dashboard

The Admin Dashboard allows authorized users to manage flight data, monitor system activity, and control notifications and updates within the platform.

Benefits of the Main Features

- Real-time flight tracking
 - Easy flight search and access
 - Interactive map-based visualization
 - Accurate and updated flight information
 - User-friendly interface
 - Instant notifications and alerts
 - Efficient flight history management
 - Secure administrative control
-

7.2 How Each Feature Works

This section explains how each feature functions from both the user's perspective and the technical implementation perspective.

1. User Authentication

UserPerspective:

The administrator logs in using valid credentials to access the Admin Dashboard. Unauthorized users cannot access admin features.

TechnicalDescription:

Authentication is implemented using components such as `AuthForm.jsx` and `AdminLoginGate.jsx`. Credentials are verified on the frontend before granting access to protected routes.

2. Flight Search

UserPerspective:

Users enter a flight number, airline name, or route to search for a specific flight. Matching results are displayed instantly.

TechnicalDescription:

The `FlightSearch.jsx` component filters flight data using user input and updates results using React state management.

3. Flight Status Display

UserPerspective:

Users can view real-time flight status such as on-time, delayed, cancelled, boarding, or landed.

TechnicalDescription:

The `FlightStatusCard.jsx` component fetches and displays live flight status data from an external Flight API.

4. Flight Details View

UserPerspective:

Users can view complete flight details like departure time, arrival time, gate, terminal, and duration.

TechnicalDescription:

The `FlightDetails.jsx` component renders detailed flight information received from API responses.

5. Interactive Map Visualization

UserPerspective:

Users can view flight routes and airport locations on an interactive map with zoom and navigation options.

TechnicalDescription:

The MapView.jsx component uses Leaflet Maps with OpenStreetMap to render flight routes and airport markers.

6. Live Flight Tracking

User Perspective:

Users can track flights in real time and see updates without refreshing the page.

TechnicalDescription:

The LiveUpdateService.jsx fetches flight data periodically from APIs and updates the UI dynamically.

7. Notification System

UserPerspective:

Users receive alerts for delays, cancellations, gate changes, and other flight updates.

TechnicalDescription:

The Notifications.jsx component displays real-time updates using React state and API-triggered events.

8. Flight History

UserPerspective:

Users can view previously searched flights for quick access.

TechnicalDescription:

The FlightHistory.jsx component stores and retrieves search history using local state or browser storage.

9. Help and Support

UserPerspective:

Users can access guidance, instructions, and FAQs for using the system.

TechnicalDescription:

The HelpSupport.jsx component displays static help content using reusable React components.

10. Administrative Dashboard

UserPerspective:

Admins can manage flight information, monitor system activity, and control notifications.

TechnicalDescription:

The AdminDashboard.jsx component provides administrative controls, while AdminLoginGate.jsx restricts access to authorized users only.

Conclusion

The Flight Status Tracker System integrates flight search, live tracking, notifications, mapping, and admin control into a single ReactJS-based application. By using external Flight APIs and Leaflet Maps with OpenStreetMap, the system delivers a responsive, real-time, and user-friendly flight tracking experience.

CHAPTER -8 TESTING

8.1 Postman Testing (Frontend/Backend)

Software testing is one of the most important phases of the Software Development Life Cycle (SDLC). It is a systematic process of evaluating and verifying that a software application performs according to specified requirements and functions correctly under various conditions. The main objective of testing is to identify defects, improve software quality, ensure reliability, and confirm that the final system meets user expectations.

Testing plays a crucial role in ensuring the functionality, accuracy, efficiency, usability, and overall performance of a software system. It helps developers detect issues early, improve system stability, and enhance the overall user experience before deployment.

The **Flight Status Tracker System** was developed to provide real-time flight tracking, flight search, status updates, notifications, map visualization, and administrative management through an interactive web-based platform. Since the system handles dynamic flight data and real-time updates, it is essential to ensure that all modules function correctly and consistently.

Testing was performed throughout the development process to verify both individual components and overall system behavior. Each feature was evaluated to ensure that users could search flights, view live status updates, access flight details, and interact with the system without errors.

Special attention was given to API integration testing, flight data retrieval, response accuracy, notification updates, map rendering, and user interface responsiveness. Postman was used to test API requests and responses, ensuring that flight data is correctly fetched and displayed in the frontend application.

The testing process also ensured compatibility across different browsers and devices. Since the application is designed as a responsive web platform, it was necessary to verify that all components function properly on various screen sizes and operating environments.

The successful completion of testing confirmed that the Flight Status Tracker System meets its functional requirements and provides a reliable, efficient, and user-friendly solution for real-time flight information.

Importance of Testing

Testing is important because it:

- Identifies software defects and errors
- Ensures accurate real-time flight data retrieval
- Improves application reliability and performance
- Enhances user experience and satisfaction
- Validates API responses and system functionality
- Reduces maintenance and debugging costs
- Ensures system stability under different conditions
- Improves overall software quality
- Confirms correct integration of external flight APIs
- Supports successful deployment of the system

8.2 Integration Testing

A testing strategy defines the overall approach used to verify and validate software functionality. It outlines the methods, procedures, and techniques used to ensure that the application operates according to its design specifications. A well-structured testing strategy helps improve software quality, reliability, and reduces the chances of system failures after deployment.

For the **Flight Status Tracker System**, a comprehensive testing strategy was adopted to verify the functionality of all major modules, including flight search, flight status display, flight details view, live updates, map visualization, notifications, authentication, and administrative components.

Since the application is developed as a frontend-based ReactJS web application without a dedicated backend server, testing mainly focused on user interface behavior, API integration, module interaction, browser compatibility, performance, and overall user experience.

The testing strategy was divided into several phases to ensure complete system validation.

Functional Testing

Functional testing was performed to verify that each feature works according to the requirements. Every module was tested individually to ensure correct output for valid inputs.

The following functionalities were tested:

- Flight search operations
- Real-time flight status display
- Flight details retrieval
- Notification system
- Map visualization
- Authentication functionality
- Admin dashboard operations

The results confirmed that all modules performed their intended functions successfully.

User Interface Testing

UI testing was conducted to ensure that all components are displayed correctly and provide a smooth user experience.

UI testing focused on:

- Button functionality
- Search bar behavior
- Layout consistency
- Responsive design
- Flight cards and dashboards
- Map display components

The testing confirmed that the application provides a clean, intuitive, and user-friendly interface.

Integration Testing

Integration testing was performed to verify communication between different modules of the system. It ensured that data flows correctly from one module to another without errors.

Examples include:

- Flight search module integration with flight status display
- Flight status module integration with notification system
- Flight details module integration with map visualization
- Authentication module integration with admin dashboard

Successful integration ensured smooth workflow across the system.

Browser Compatibility Testing

Browser compatibility testing was conducted to ensure consistent performance across different browsers.

The application was tested on:

- Google Chrome
- Mozilla Firefox
- Microsoft Edge

The results confirmed that all features such as flight search, maps, and live updates work properly across all browsers.

Performance Testing

Performance testing was conducted to evaluate system responsiveness and efficiency under normal conditions.

Key parameters tested:

- API response time for flight data
- Page loading speed
- Map rendering performance
- Real-time update responsiveness
- UI interaction speed

The system showed smooth performance and fast response times.

Module Testing

Each module was tested individually before integration into the complete system.

Modules tested include:

- Authentication Module
- Flight Search Module
- Flight Status Module
- Flight Details Module
- Live Update Module
- Map Visualization Module
- Notification Module
- Flight History Module
- Help and Support Module
- Administrative Module

This ensured early detection and correction of errors during development.

Testing Environment

Component	Specification
Operating System	Windows 10/11
Browser	Chrome, Firefox, Edge
Frontend Framework	ReactJS
Build Tool	Vite
Mapping Technology	Leaflet Maps
Map Service	OpenStreetMap

Benefits of the Testing Strategy

The adopted testing strategy provided several advantages:

- Improved software quality
 - Better system reliability
 - Early detection of errors
 - Smooth API integration
 - Enhanced user experience
-

- Faster system response
- Verified module communication
- Stable and consistent performance
- Successful integration of all features

The testing strategy ensured that the **Flight Status Tracker System** operates efficiently, accurately, and reliably while providing a seamless real-time flight tracking experience for users.

8.3 Tools Used for Testing

The testing of the **Flight Status Tracker System** was carried out using various software tools and development environments to verify functionality, performance, responsiveness, and reliability of the application. These tools helped in identifying errors, debugging issues, monitoring application behaviour, and ensuring smooth operation of all modules.

Since the project is developed as a frontend-based ReactJS web application, testing mainly focused on user interface behavior, flight search functionality, API integration, real-time updates, map visualization, browser compatibility, and overall system responsiveness.

The following tools were used during the testing process.

Google Chrome

Google Chrome was the primary browser used for testing the application. It was used to verify flight search functionality, UI behavior, real-time updates, and map-based visualization.

Purpose:

- Functional testing
- User interface testing
- Browser compatibility testing
- Performance verification

Features:

- Developer tools support
- JavaScript execution monitoring
- Console error tracking
- Network request inspection

Chrome Developer Tools

Chrome Developer Tools (DevTools) were used to debug API calls, inspect network requests, and monitor frontend performance during testing.

Purpose:

- Debugging React components
- Monitoring API responses
- Error detection
- Performance analysis

Features:

- Console debugging
- Network monitoring
- Element inspection
- Performance profiling

Visual Studio Code

Visual Studio Code was used for writing, debugging, and testing the application code during development.

Purpose:

- Code verification
- Error identification
- Application debugging
- Source code management

Features:

- Integrated terminal
- Debugging support
- Extensions for React
- Code linting and validation

React Developer Tools

React Developer Tools were used to inspect React components and monitor state changes during testing.

Purpose:

- Component inspection
- State and props monitoring
- React debugging
- Performance optimization

Features:

- Component hierarchy view
 - State tracking
 - Props inspection
 - Debugging support
-

Vite Development Server

The Vite development server was used to run and test the application in a local environment.

Purpose:

- Local application testing
- Module integration testing
- Real-time updates

Features:

- Fast build and startup
 - Hot Module Replacement (HMR)
 - Quick debugging support
-

Flight Data API Testing Tools (Postman / Console Testing)

API testing was performed using Postman and browser console to verify flight data responses.

Purpose:

- Validate API responses
- Test flight search queries
- Verify real-time data accuracy

Features:

- API request/response testing
 - JSON data inspection
 - Error handling verification
-

Browser Geolocation Services

Geolocation services were tested to ensure correct location detection (if used for nearby airport features).

Purpose:

- Location tracking validation
- Navigation testing
- Coordinate verification

Features:

- Real-time location access
 - Latitude and longitude detection
 - Position accuracy testing
-

Testing Tools Summary

Tool	Purpose
Google Chrome	Functional and UI Testing
Chrome Developer Tools	Debugging and Performance Analysis
Visual Studio Code	Code Development and Debugging
React Developer Tools	Component and State Testing
Vite Development Server	Local Application Testing
Postman / Console	API Testing
Geolocation Services	Location Testing

Conclusion

Various testing tools were used to ensure the quality and reliability of the **Flight Status Tracker System**. These tools helped identify and resolve issues related to API integration, user interface behavior, flight search functionality, and system performance. The use of these tools significantly contributed to the successful development and validation of the application.

8.4 Test Cases and Results

Test cases are designed to verify whether the functionalities of the **Flight Status Tracker System** operate according to the specified requirements. Each test case includes input conditions, expected results, and actual outcomes obtained during testing. The purpose of executing test cases is to ensure that all modules function correctly and deliver accurate real-time flight information. The testing process was conducted on key modules of the system, including flight search, flight status display, flight details, live updates, map visualization, notifications, authentication, and administrative functions. The results confirmed that the system meets all functional requirements and provides a reliable user experience.

Test Case 1: User Login Verification

Field	Description
Test Case ID	TC-01
Module	Authentication Module
Test Objective	Verify user login functionality
Input	Valid admin credentials
Expected Result	Admin should access dashboard successfully
Actual Result	Login successful
Status	Pass

Description:
This test verifies that the authentication system correctly validates user credentials and allows authorized access.

Test Case 2: Flight Search Functionality

Field	Description
Test Case ID	TC-02
Module	Flight Search Module
Test Objective	Verify flight search operation
Input	Flight number or airline name
Expected Result	Relevant flight details should be displayed
Actual Result	Flight data displayed correctly
Status	Pass

Description:
This test ensures that users can successfully search flights using valid input parameters.

Test Case 3: Flight Status Display

Field	Description
Test Case ID	TC-03
Module	Flight Status Module
Test Objective	Verify real-time status display
Input	Flight selection
Expected Result	Live status (on-time/delayed/cancelled) should appear
Actual Result	Status displayed correctly
Status	Pass

Description:
This test validates that real-time flight status updates are displayed accurately.

Test Case 4: Map Visualization

Field	Description
Test Case ID	TC-04
Module	Map Visualization Module
Test Objective	Verify flight route display
Input	Select flight route
Expected Result	Flight path should be shown on map
Actual Result	Map rendered successfully
Status	Pass

Description:
This test ensures correct integration of Leaflet Maps and OpenStreetMap for route visualization.

Test Case 5: Live Flight Updates

Field	Description
Test Case ID	TC-05
Module	Live Update Module
Test Objective	Verify real-time data updates
Input	Active flight tracking
Expected Result	Flight status should update automatically
Actual Result	Live updates working
Status	Pass

Description:
This test validates periodic API calls for real-time flight tracking.

Test Case 6: Flight Details View

Field	Description
Test Case ID	TC-06
Module	Flight Details Module
Test Objective	Verify detailed flight information display
Input	Select flight
Expected Result	Full flight details should be shown
Actual Result	Details displayed correctly
Status	Pass

Description:

This test ensures complete flight information is properly rendered.

Test Case 7: Notification System

Field	Description
Test Case ID	TC-07
Module	Notification Module
Test Objective	Verify notification alerts
Input	Flight delay or update event
Expected Result	Notification should appear
Actual Result	Notification displayed successfully
Status	Pass

Description:

This test checks if users receive real-time flight alerts.

Test Case 8: Flight History

Field	Description
Test Case ID	TC-08
Module	Flight History Module
Test Objective	Verify history storage functionality
Input	Multiple flight searches
Expected Result	Previous searches should be saved
Actual Result	History stored correctly
Status	Pass

Description:

This test ensures that previously searched flights are stored and accessible.

Test Case 9: Admin Dashboard Access

Field	Description
Test Case ID	TC-09
Module	Administrative Module
Test Objective	Verify admin panel access
Input	Valid admin login
Expected Result	Admin dashboard should open
Actual Result	Dashboard accessed successfully
Status	Pass

Description:

This test confirms secure access to administrative functionalities.

Overall Testing Results

The execution of test cases confirmed that all major functionalities of the **Flight Status Tracker System** performed successfully. Flight search, live tracking, status updates, map visualization, notifications, and administrative operations all functioned as expected.

Test Summary

Total Test Cases Passed Failed

9	9	0
---	---	---

Conclusion

The testing results indicate that the **Flight Status Tracker System** is functioning correctly and meets all specified requirements. All modules passed successfully, confirming the reliability, accuracy, and usability of the system. The application is ready for deployment and future enhancements.

CHAPTER -9 DEPLOYMENT

9.1 Steps to Deploy

Deployment is the process of making the **Flight Status Tracker System** available for users to access and use. Since the project is developed as a frontend-based web application using **ReactJS and Vite**, deployment is simple and does not require a backend server or database setup.

The following steps describe the deployment procedure used for the project.

Step 1: Install Node.js

Node.js must be installed on the system to run the application and manage dependencies.

Procedure:

- Download Node.js from the official website.
- Install Node.js and npm (Node Package Manager).
- Verify installation using:

```
node -v
```

```
npm -v
```

Successful installation ensures the environment is ready for project execution.

Step 2: Obtain the Project Source Code

The project source code must be available on the local system.

Procedure:

- Download or clone the project folder.
- Extract files if compressed.
- Open the project in Visual Studio Code.

This prepares the project for setup and execution.

Step 3: Install Dependencies

The project uses ReactJS libraries and external packages that must be installed.

Procedure:

Open the terminal in the project directory and run:

```
npm install
```

This installs all required dependencies from the package.json file.

Step 4: Start the Development Server

The application is executed using the Vite development server.

Procedure:

Run the command:

```
npm run dev
```

The server starts and provides a local URL such as:

```
http://localhost:5173
```

Step 5: Access the Application

Open the application in a web browser.

Procedure:

- Copy the local URL.
- Paste it into Chrome, Firefox, or Edge.
- Press Enter.

The Flight Status Tracker System home page will load successfully.

Step 6: Use Flight Tracking Features

After deployment, users can access all system features.

Procedure:

- Search for flights
- View real-time flight status
- Check flight details

- View map-based route visualization
- Receive notifications and updates

Step 7: Verify Application Functionality

All modules should be tested after deployment.

Verification Includes:

- Flight search functionality
- Live flight status updates
- Map visualization
- Notification system
- Admin dashboard access

Successful execution confirms proper deployment.

Deployment Environment

Component	Configuration
Operating System	Windows 10/11
Frontend Framework	ReactJS
Build Tool	Vite
Programming Language	JavaScript
Map Library	Leaflet
Map Service	OpenStreetMap
Browser	Chrome / Firefox / Edge
Runtime Environment	Node.js

Conclusion

The deployment process of the **Flight Status Tracker System** is simple and efficient because it is implemented as a frontend-only ReactJS application. By installing Node.js, setting up dependencies, running the Vite server, and accessing the application through a browser, the system can be successfully deployed.

This lightweight architecture eliminates the need for backend configuration, making the application easy to install, run, and maintain while ensuring fast performance and smooth user experience.

9.2 Environment Configuration

Environment configuration refers to the setup of software tools, libraries, frameworks, and system settings required for the successful execution of the **Flight Status Tracker System**. Proper configuration ensures that the application runs smoothly during development, testing, and deployment.

The Flight Status Tracker System is developed using **ReactJS and Vite** as a frontend-based web application. Therefore, the environment mainly consists of development tools, runtime setup, APIs, and browser support required for proper execution.

Operating System Configuration

The application can be developed and executed on modern operating systems that support Node.js and web browsers.

Supported Operating Systems:

- Windows 10
- Windows 11
- Linux
- macOS

These systems provide a stable platform for running development tools and the ReactJS application.

Node.js Configuration

Node.js is required to execute JavaScript code outside the browser and manage dependencies.

Configuration Steps:

- Install Node.js from the official website
- Verify installation using:

node -v

Purpose:

- Provides runtime environment
 - Supports JavaScript execution
 - Manages application dependencies
-

npm Configuration

npm (Node Package Manager) is used to install and manage project libraries.

Configuration Command:

npm install

Purpose:

- Installs required packages
 - Manages project dependencies
 - Maintains consistent project setup
-

ReactJS Environment

ReactJS is the primary framework used for building the user interface of the application.

Configuration:

All React libraries are installed automatically using npm during setup.

Purpose:

- Builds interactive UI
 - Supports component-based architecture
 - Handles state management
 - Enables dynamic rendering of flight data
-

Vite Configuration

Vite is used as the development server and build tool.

Configuration Command:

npm run dev

Purpose:

- Fast project execution
 - Hot Module Replacement (HMR)
 - Efficient development workflow
 - Optimized builds
-

Browser Configuration

A modern web browser is required to run and test the application.

Supported Browsers:

- Google Chrome
- Mozilla Firefox
- Microsoft Edge

Purpose:

- Runs the web application
 - Displays flight data and UI
 - Supports API requests
 - Enables interactive features
-

API Configuration

The system integrates external APIs for real-time flight data.

Configuration Requirements:

- Flight Data API key (if required)
- Proper API endpoint setup in code
- Internet connectivity

Purpose:

- Fetch real-time flight status
 - Retrieve flight details
 - Enable live tracking updates
-

Leaflet and OpenStreetMap Configuration

Leaflet Maps and OpenStreetMap are used for flight route visualization.

Configuration:

- Installed via npm
-

- Integrated into React components

Purpose:

- Display flight routes
- Show airport locations
- Provide interactive maps
- Support zoom and navigation

Environment Configuration Summary

Component	Configuration
Operating System	Windows 10/11, Linux, macOS
Runtime Environment	Node.js
Package Manager	npm
Frontend Framework	ReactJS
Build Tool	Vite
Mapping Library	Leaflet
Map Service	OpenStreetMap
Browser Support	Chrome, Firefox, Edge
API Integration	Flight Data API

Conclusion

The environment configuration for the **Flight Status Tracker System** is simple and efficient due to its frontend-based architecture. By setting up Node.js, npm, ReactJS, Vite, external flight APIs, and browser support, the application can be successfully developed, tested, and deployed.

This setup ensures smooth execution, real-time flight data handling, and a responsive user experience across different devices.

9.3 Hosting of Frontend and Backend

Hosting refers to the process of making an application available to users through a server or cloud platform. The **Flight Status Tracker System** is developed as a frontend-based web application using **ReactJS and Vite**. The current implementation focuses mainly on frontend functionalities and real-time API integration, without a dedicated backend server or database system.

Frontend Hosting

The frontend of the application includes ReactJS components such as flight search, flight status display, flight details view, live updates, notifications, map visualization, and admin dashboard interfaces. During development, the application is hosted locally using the Vite development server.

Frontend Hosting Procedure

- Install project dependencies using npm
- Start the Vite development server
- Access the application through a web browser
- Verify functionality of all modules

The application runs on a local host address generated by Vite, for example:

http://localhost:5173

The frontend interacts directly with external Flight Data APIs, Leaflet Maps, and OpenStreetMap services to provide real-time flight tracking and visualization.

Frontend Hosting Technologies

Component	Technology
User Interface	ReactJS
Build Tool	Vite
Styling	CSS
Mapping Library	Leaflet
Map Service	OpenStreetMap
API Integration	Flight Data API

Future Frontend Hosting Platforms

The frontend can be deployed to cloud hosting platforms for production use, such as:

- GitHub Pages
- Netlify
- Vercel
- Firebase Hosting

These platforms provide scalable, fast, and reliable hosting for ReactJS applications.

Backend Hosting

The current version of the **Flight Status Tracker System** does not include a backend server. Therefore, backend hosting is not required.

No server-side technologies such as:

- Node.js
- Express.js
- Django
- Flask
- Spring Boot

and no database systems such as:

- MySQL
- MongoDB
- PostgreSQL
- Firebase Firestore

are used in the current implementation.

All functionalities are handled through frontend components and external APIs.

Future Backend Hosting Possibilities

If backend features are added in future versions, hosting can be done using:

- AWS (Amazon Web Services)
 - Microsoft Azure
 - Google Cloud Platform (GCP)
 - Render
-

- Railway
- Firebase

These backend services can support:

- User authentication
- Flight data storage
- Real-time notifications
- Admin data management
- Secure API handling

Hosting Architecture Summary

Layer	Hosting Method
-------	----------------

Frontend	Local Vite Server / Cloud Hosting
----------	-----------------------------------

APIs	External Flight Data APIs
------	---------------------------

Maps	Leaflet + OpenStreetMap
------	-------------------------

Backend	Not Implemented
---------	-----------------

Database	Not Implemented
----------	-----------------

Conclusion

The **Flight Status Tracker System** follows a frontend-only hosting architecture. The application is hosted locally using the Vite development server and integrates external APIs for real-time flight tracking and map visualization. No backend or database hosting is required in the current version, making the system lightweight, efficient, and easy to deploy. The architecture also allows future expansion into cloud-based backend services if needed.

CHAPTER -10 CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

During the development of the **Flight Status Tracker System**, several challenges were encountered while designing, implementing, and testing different modules of the application. These challenges provided valuable learning experiences and helped improve the overall quality, performance, and reliability of the system. The issues were addressed through continuous debugging, testing, and optimization.

User Interface Design Challenges

Designing a clean, responsive, and user-friendly interface was one of the initial challenges during development. The system includes multiple modules such as flight search, flight status display, notifications, maps, and admin dashboard, which required proper layout design.

Issues Encountered:

- Maintaining consistent UI across components
- Designing responsive layouts for different screen sizes
- Organizing flight information clearly
- Ensuring smooth user experience

Resolution:

ReactJS component-based architecture and responsive CSS design techniques were used to build a consistent and user-friendly interface.

Flight API Integration Challenges

Integrating external Flight Data APIs was one of the major technical challenges in the system.

Issues Encountered:

- Inconsistent API responses
- Delayed or missing flight data
- Handling API errors and downtime
- Data formatting issues

Resolution:

Proper API handling techniques, error handling, and response validation were implemented to ensure reliable data retrieval.

Real-Time Data Update Issues

Since the system provides live flight tracking, maintaining real-time updates was challenging.

Issues Encountered:

- Delayed flight status updates
- Frequent API request limitations
- Synchronization issues between UI and API data

Resolution:

Efficient polling methods and optimized API calls were used to ensure smooth real-time updates.

Map Integration Challenges

Integrating Leaflet Maps with OpenStreetMap for flight route visualization presented several issues.

Issues Encountered:

- Incorrect route rendering
- Map loading delays
- Marker positioning errors
- API-map synchronization issues

Resolution:

Proper configuration of Leaflet and improved data mapping logic ensured accurate visualization of flight routes.

React Component Management

Managing multiple React components such as flight search, status display, notifications, and admin modules became complex.

Issues Encountered:

- Component re-rendering problems
- State management complexity
- Data flow between components
- Code organization difficulties

Resolution:

Reusable components, props handling, and structured state management were implemented to improve maintainability.

Browser Compatibility Issues

The application behaved differently across various browsers during testing.

Issues Encountered:

- UI layout inconsistencies
- Geolocation permission differences
- CSS rendering variations

Resolution:

Cross-browser testing was performed using Chrome, Firefox, and Edge to ensure consistent performance.

Debugging and Error Handling

Debugging API and frontend issues required continuous monitoring during development.

Issues Encountered:

- API errors
- Runtime JavaScript issues
- Missing or incorrect data responses
- Component integration errors

Resolution:

Chrome DevTools, console logs, and systematic debugging techniques were used to resolve issues effectively.

Performance Optimization Challenges

As the application grew, maintaining performance became important.

Issues Encountered:

- Slow rendering of flight data
-

- Increased API load time
- UI lag during updates

Resolution:

Optimized React rendering, reduced unnecessary API calls, and improved component efficiency.

Lessons Learned

The challenges faced during development provided valuable experience in:

- ReactJS frontend development
 - API integration and handling
 - Real-time data processing
 - Map visualization using Leaflet
 - Component-based architecture
 - Debugging and testing techniques
 - Performance optimization
 - Web application development practices
-

Conclusion

Although several challenges were encountered during the development of the **Flight Status Tracker System**, all major issues were successfully resolved through continuous testing, debugging, and optimization. Overcoming these challenges significantly improved system performance and enhanced understanding of modern web application development.

10.2 Solutions Applied

During the development of the Flight Status Tracker System, several technical and implementation challenges were encountered. Appropriate solutions were applied to overcome these issues and ensure successful completion of the project. The solutions focused on improving system functionality, enhancing user experience, ensuring real-time accuracy, and maintaining overall performance.

Improving User Interface Design

One of the major challenges was designing a clean and responsive interface for displaying flight information clearly.

Solution Applied:

A component-based UI was developed using ReactJS with reusable components such as flight cards, status panels, and notification modules. Consistent layout structure and responsive design techniques were applied.

Benefits:

- Improved user experience
 - Better readability of flight data
 - Consistent UI across pages
 - Easier navigation
-

Resolving Flight API Integration Issues

Integrating external Flight Data APIs initially caused inconsistent or delayed responses.

Solution Applied:

Proper API handling, error management, and response validation techniques were implemented to ensure accurate data fetching.

Benefits:

- Reliable flight data retrieval
 - Reduced API errors
 - Improved system stability
-

Enhancing Real-Time Flight Updates

Maintaining real-time flight tracking and updates was a major challenge.

Solution Applied:

Efficient polling mechanisms and optimized API calls were used to update flight status dynamically.

Benefits:

- Up-to-date flight information
 - Faster status updates
 - Improved tracking accuracy
-

Improving Map Visualization

Integration of Leaflet Maps and OpenStreetMap for flight routes required optimization.

Solution Applied:

Proper configuration of map components and accurate mapping of flight coordinates were implemented.

Benefits:

- Accurate flight route visualization
 - Smooth map rendering
 - Better user understanding
-

Managing React Component Complexity

As the system expanded, managing multiple components became difficult.

Solution Applied:

A modular architecture was implemented with reusable React components and proper state management techniques.

Benefits:

- Improved code maintainability
 - Easier debugging
 - Better scalability
 - Reduced code duplication
-

Browser Compatibility Improvements

Different browsers displayed UI elements differently.

Solution Applied:

Cross-browser testing was conducted using Chrome, Firefox, and Edge, and necessary adjustments were made.

Benefits:

- Consistent user experience
 - Improved compatibility
 - Better system reliability
-

Error Detection and Debugging

Several runtime and integration errors were encountered during development.

Solution Applied:

Chrome DevTools, React Developer Tools, and VS Code debugging tools were used for systematic error detection and resolution.

Benefits:

- Faster debugging
- Reduced errors
- Improved system stability

Performance Optimization

As more features were added, maintaining performance became important.

Solution Applied:

Optimized React rendering, reduced unnecessary API calls, and improved component efficiency.

Benefits:

- Faster application response
- Reduced loading time
- Smooth user experience

Issue and Solution Summary

Issue Faced

UI complexity

API integration issues

Real-time update delays

Map visualization errors

Component complexity

Browser inconsistencies

Debugging difficulties

Performance issues

Solution Applied

React component-based design

Proper API handling and validation

Optimized polling and API calls

Leaflet + OSM configuration

Modular architecture

Cross-browser testing

Developer tools usage

Code optimization

Conclusion

The challenges encountered during the development of the Flight Status Tracker System were successfully resolved through structured problem-solving, continuous testing, and optimized coding practices. The solutions implemented improved system performance, enhanced user experience, and ensured reliable real-time flight tracking functionality.

10.3 Current Limitations or Known Bugs

Although the **Flight Status Tracker System** successfully implements its core functionalities such as flight search, real-time status display, notifications, and map visualization, certain limitations and minor issues exist in the current version of the application. These limitations arise mainly due to the frontend-only architecture, API dependency, and project scope constraints. Identifying these limitations helps in understanding areas for future improvement.

Absence of Backend Integration

The current version of the system is implemented as a frontend-only application without a dedicated backend server.

Limitation:

- No server-side processing
- No centralized data management
- No secure API handling layer
- Limited scalability for large-scale usage

Impact:

Advanced features such as secure user authentication, cloud data storage, and server-side processing cannot be implemented in the current version.

No Database Management System

The system does not use any database for storing flight history, user data, or system logs.

Limitation:

- No persistent data storage
- Flight search history is limited to session-based storage
- No long-term data tracking

Impact:

User data and search history are not retained after refreshing or closing the application.

Dependency on Internet Connectivity

The application relies heavily on external Flight Data APIs and map services.

Limitation:

- Requires stable internet connection
- Flight data may not load offline
- Map services may fail under poor network conditions

Impact:

Users may experience delays or inability to access real-time flight information in low connectivity areas.

Flight API Limitations

The system depends on third-party flight data providers.

Limitation:

- API rate limits may restrict frequent requests
- Occasional delayed or incomplete flight data
- Dependency on external service availability

Impact:

Real-time accuracy may be affected during high traffic or API downtime.

Geolocation Accuracy Limitations

The system uses browser-based geolocation services.

Limitation:

- Accuracy depends on device and browser
- GPS availability affects precision
- Requires user permission

Impact:

Location-based features may show slight variations in accuracy.

Limited Real-Time Features

The current implementation provides basic real-time updates but lacks advanced synchronization.

Limitation:

- No multi-user live synchronization
- No real-time server push notifications
- Limited live tracking optimization

Impact:

The system cannot support advanced airline-grade real-time monitoring features.

Browser Dependency

Some functionalities depend on browser support and behavior.

Limitation:

- Performance varies across browsers
- Differences in rendering UI components
- Geolocation permission handling varies

Impact:

Minor inconsistencies may appear between Chrome, Firefox, and Edge.

Limited Administrative Features

The admin module provides only basic functionality.

Limitation:

- No backend-based admin control system
- Limited analytics and reporting
- No advanced data management tools

Impact:

Administrative capabilities are basic compared to production-level systems.

Known Bugs

During testing, only minor issues were observed, which do not significantly affect system functionality.

Minor Issues Identified:

- Occasional delay in flight data loading
- Temporary lag in map rendering
- Slight UI misalignment on small screens
- Delay in updating flight status after API refresh

Severity:

These issues are minor and do not affect the core functionality of the system.

Future Improvements

The following enhancements can address the current limitations:

- Integration of a backend server
 - Use of a database for storing flight history
-

- Improved real-time flight tracking system
 - Advanced API integration with airline systems
 - Cloud deployment for scalability
 - Enhanced admin dashboard with analytics
 - Offline caching for basic flight information
 - Improved performance optimization
-

Conclusion

The **Flight Status Tracker System** successfully meets its primary objectives despite certain limitations and minor issues. Most limitations are due to its frontend-only architecture and dependency on external APIs. These limitations provide clear opportunities for future enhancements, making the system scalable and more robust in future versions.

CHAPTER -11 FUTURE ENHANCEMENTS

11.1 Planned Features

The **Flight Status Tracker System** is designed to provide an efficient, user-friendly, and real-time solution for tracking flight information. The system integrates flight search, live status updates, route visualization, notifications, and administrative functionalities into a single web-based platform. The planned features focus on improving accessibility, enhancing real-time tracking, and providing accurate flight information to users.

The major planned features of the system are described below:

User Authentication

The platform includes a secure login system for administrators. Authentication ensures controlled access to administrative functionalities and helps maintain system security.

Flight Search Functionality

Users can search for flights using flight number, airline name, or route information. This feature allows quick access to specific flight details.

Real-Time Flight Tracking

The system provides live flight status updates including departure time, arrival time, delays, cancellations, and gate information.

Interactive Map Visualization

The application uses Leaflet Maps and OpenStreetMap services to display flight routes and aircraft movement on an interactive map interface.

Flight Route Guidance

The system visually displays flight paths between departure and destination airports, helping users understand flight movement and route progress.

Airport Information Display

Users can view detailed information about airports such as terminal details, arrival/departure boards, and flight schedules.

Notification System

The platform includes a notification feature to alert users about flight delays, cancellations, gate changes, and important updates.

Emergency and Delay Alerts

The system highlights urgent updates such as flight delays, cancellations, or emergency announcements to ensure timely communication.

Help and Support Services

A help section is provided to guide users on how to search flights, track status, and use different features of the application.

Administrative Dashboard

The administrative dashboard allows authorized users to monitor system activity, manage flight data display settings, and oversee application operations.

Responsive User Interface

The application is designed using responsive web technologies to ensure compatibility across desktops, tablets, and mobile devices, providing a smooth user experience across all screen sizes.

Planned Features Summary

- User Authentication
 - Flight Search Functionality
 - Real-Time Flight Tracking
 - Interactive Map Visualization
 - Flight Route Display
 - Airport Information System
 - Notification System
 - Delay and Emergency Alerts
 - Help and Support Services
 - Administrative Dashboard
 - Responsive User Interface
-

Conclusion

These planned features collectively contribute to the development of a comprehensive **Flight Status Tracker System** capable of providing real-time flight information, improving user experience, and enhancing accessibility for passengers and users. The system is designed to be scalable and can be further enhanced with advanced analytics, AI-based predictions, and global flight integration in future versions.

11.2 Possible Integrations or Optimizations

The current version of the **Flight Status Tracker System** successfully provides flight search, real-time status updates, flight details display, notifications, and map visualization. However, several integrations and optimizations can be implemented in future versions to improve performance, scalability, accuracy, and user experience. These enhancements can transform the system into a more advanced and intelligent flight tracking platform.

Backend Integration

The current system is frontend-based. Adding a backend server would significantly enhance functionality.

Possible Integrations:

- Node.js and Express.js
- Django Framework
- Flask Framework
- Spring Boot

Benefits:

- Server-side processing
 - Centralized data handling
 - Improved scalability
 - Enhanced security
-

Database Integration

A database system can be used to store flight-related and user-related data efficiently.

Possible Databases:

- MySQL
- PostgreSQL
- MongoDB
- Firebase Firestore

Benefits:

- Permanent data storage
 - User search history tracking
 - Flight bookmarking system
 - Administrative data management
-

Real-Time Flight Tracking Enhancements

The system can be improved to provide more accurate and continuous real-time updates.

Possible Technologies:

- WebSockets
- Socket.IO
- Firebase Real-Time Database

Benefits:

- Instant flight status updates
 - Reduced API polling delays
 - Improved user experience
-

Cloud Deployment

Deploying the system on cloud platforms can improve accessibility and reliability.

Possible Platforms:

- Amazon Web Services (AWS)
- Microsoft Azure
- Google Cloud Platform (GCP)
- Vercel
- Firebase Hosting

Benefits:

- High availability
 - Better scalability
 - Remote access from anywhere
 - Improved performance
-

Advanced Flight Route Optimization

Flight tracking can be enhanced with smarter route analysis and visualization.

Possible Improvements:

- Real-time flight path prediction
-

- Weather-based route adjustments
- Distance and duration optimization
- Multi-airport route comparison

Benefits:

- Better flight insights
 - Improved accuracy
 - Enhanced visualization
-

Artificial Intelligence Integration

AI can be used to improve prediction and user assistance features.

Possible Applications:

- Flight delay prediction
- Smart flight recommendations
- Passenger travel insights
- Pattern analysis of flight data

Benefits:

- Intelligent predictions
 - Improved decision-making
 - Enhanced user experience
-

Mobile Application Development

The system can be extended into mobile applications for better accessibility.

Possible Platforms:

- Android
- iOS
- Cross-platform (Flutter / React Native)

Benefits:

- Improved accessibility
 - On-the-go flight tracking
 - Better user convenience
-

Offline Support and Caching

Currently, the system requires internet connectivity.

Optimizations:

- Cached flight data storage
- Offline mode for recent searches
- Local storage support

Benefits:

- Improved usability in low-network areas
 - Faster access to recent data
-

- Reduced API dependency
-

Performance Optimization

The application can be optimized for better speed and efficiency.

Possible Optimizations:

- Lazy loading of components
- Code splitting
- API request optimization
- Efficient state management

Benefits:

- Faster loading times
 - Reduced resource usage
 - Smooth user experience
-

Security Enhancements

Future versions can include stronger security measures.

Possible Enhancements:

- Secure authentication system (JWT)
- Role-based access control
- Encrypted API communication
- Secure user sessions

Benefits:

- Improved data protection
 - Safer user interactions
 - Increased system reliability
-

Analytics and Reporting

Advanced analytics can provide insights into system usage and flight trends.

Possible Features:

- Flight search analytics
- Delay trend analysis
- User activity tracking
- Admin dashboards with reports

Benefits:

- Better decision-making
 - System monitoring
 - Improved service optimization
-

Conclusion

The **Flight Status Tracker System** provides a strong foundation for real-time flight tracking and information display. Future

integrations such as backend systems, databases, cloud deployment, AI-based predictions, mobile applications, and advanced analytics can significantly enhance the platform. These optimizations will improve scalability, accuracy, performance, and user experience, making the system more powerful and production-ready.

CHAPTER -12 CONCLUSION

12.1 Summary of the Project

The **Flight Status Tracker System** is a web-based application developed to provide real-time flight information and improve accessibility to accurate flight status updates for users. The project aims to simplify the process of tracking flights, checking schedules, and receiving live updates on delays, cancellations, and gate changes. By combining modern web technologies with external flight data services and interactive mapping tools, the platform offers a user-friendly solution for efficient flight tracking.

The application was developed using **ReactJS** as the frontend framework and **Vite** as the development environment. Interactive maps were implemented using **Leaflet Maps**, while geographical visualization was supported using **OpenStreetMap services**. External Flight Data APIs were integrated to fetch real-time flight information such as departure times, arrival times, delays, and flight status updates.

The platform consists of several modules including flight search, real-time flight tracking, route visualization, airport information display, notifications, emergency alerts, help and support services, and administrative functionalities. These modules work together to provide users with accurate and timely flight information.

A modular development approach was adopted to ensure maintainability, scalability, and efficient code organization. The application was thoroughly tested to verify functionality, responsiveness, browser compatibility, API integration, and overall performance. The testing results demonstrated that all major features operate successfully and meet the intended project objectives.

One of the key advantages of the project is its lightweight architecture. Since the current implementation is frontend-based, it does not require a dedicated backend server or database system. This simplifies deployment while still effectively demonstrating real-time flight tracking and information display functionalities.

The project successfully achieves its primary goals of providing real-time flight status updates, improving accessibility to flight information, and offering an intuitive user interface for travelers and users. Through the integration of APIs and mapping technologies, the platform delivers an effective and practical solution for modern flight tracking needs.

In conclusion, the **Flight Status Tracker System** represents a practical and efficient approach to real-time flight monitoring. The project demonstrates the effective use of web technologies to solve real-world problems and provides a strong foundation for future enhancements such as backend integration, database support, AI-based predictions, cloud deployment, and advanced analytics systems.

.

12.2 What Was Achieved

The development of the **Flight Status Tracker System** successfully achieved the objectives defined during the planning and design phases of the project. The system was developed as a functional web-based application that provides real-time flight tracking, flight status updates, airport information, and interactive map-based visualization for users.

The project demonstrates how modern web technologies can be effectively used to improve access to accurate flight information and enhance the user experience. Through the integration of **ReactJS, Leaflet Maps, OpenStreetMap, and external Flight Data APIs**, the platform provides an interactive and user-friendly environment for flight tracking and information retrieval.

The following achievements were accomplished during the project development:

Successful Development of a Web-Based Flight Tracking Platform

A fully functional web application was developed to allow users to search flights and view real-time flight status information in an easy and efficient manner.

Interactive Map Implementation

Interactive maps were successfully integrated using **Leaflet Maps and OpenStreetMap services**, allowing users to view flight routes, airport locations, and aircraft movement visually.

Real-Time Flight Data Integration

External **Flight Data APIs** were successfully integrated to fetch real-time information such as departure time, arrival time, delays, cancellations, and gate updates.

Flight Route Visualization System

The platform successfully displays flight routes between departure and destination airports, helping users understand flight paths and movement clearly.

Flight Search Functionality

Users can search for flights using flight number, airline name, or route details through an efficient search system.

Notification and Alert System

The system successfully provides notifications and alerts for flight delays, cancellations, and important updates, improving user awareness.

Airport Information Display

Airport-related details such as terminal information, schedules, and flight boards are successfully displayed within the application.

Administrative Features

Basic administrative modules were implemented to manage and monitor flight-related data display and system functionality.

Responsive User Interface

A responsive and user-friendly interface was successfully developed, ensuring compatibility across desktops, laptops, tablets, and mobile devices.

Modular System Architecture

The application was built using a modular **ReactJS architecture**, improving code organization, maintainability, and scalability.

Successful Testing and Validation

All major modules were tested successfully, and results confirmed that the system functions correctly according to the defined requirements.

Major Achievements Summary

- Developed a complete frontend-based flight tracking application
 - Implemented real-time flight status display
 - Integrated Flight Data APIs successfully
 - Implemented interactive map visualization
 - Developed flight search and filtering system
 - Implemented notifications and alert system
 - Created airport information display module
 - Designed a responsive and user-friendly UI
 - Successfully tested all major functionalities
 - Achieved modular and scalable architecture
-

Conclusion

The **Flight Status Tracker System** successfully achieved its primary objectives by providing an efficient and real-time solution for flight tracking and information access. The project demonstrates the practical application of modern web technologies in solving real-world travel-related challenges and establishes a strong foundation for future enhancements such as backend integration, AI-based predictions, cloud deployment, and advanced analytics systems.

12.3 Skills Learned During Development

The development of the **Flight Status Tracker System** provided valuable learning opportunities in various areas of software development, web technologies, API integration, testing, debugging, and project management. Throughout the project lifecycle, important technical and practical skills were acquired during the design, implementation, testing, and documentation of the system. The project helped in gaining hands-on experience with modern frontend development technologies and improved understanding of real-world software development practices. It also enhanced analytical thinking, debugging abilities, and problem-solving skills required for building real-time applications.

The major skills learned during the development process are described below.

Frontend Development Skills

One of the most important skills acquired was frontend development using **ReactJS**.

Skills Learned:

- ReactJS component development
- JSX syntax usage
- State and props management
- Component-based architecture
- Dynamic UI creation
- Responsive web design

Benefits:

These skills improved the ability to develop modern, interactive, and scalable web applications.

JavaScript Programming Skills

The project strengthened understanding of JavaScript and its real-world applications.

Skills Learned:

- Event handling
- Function implementation
- API data handling
- Conditional logic
- Error handling
- Data processing

Benefits:

Improved programming logic and problem-solving capabilities.

API Integration Skills

Working with Flight Data APIs provided practical experience in real-time data handling.

Skills Learned:

- API request handling (fetch/axios)
- JSON data processing
- Error and response handling
- Real-time data integration

Benefits:

Enhanced understanding of external service integration and live data systems.

Mapping and Visualization Skills

The project involved integration of mapping technologies for flight route visualization.

Skills Learned:

- Leaflet Maps integration
- OpenStreetMap usage
- Route visualization
- Marker placement
- Interactive map handling

Benefits:

Improved knowledge of geospatial visualization and mapping systems.

User Interface and UX Design Skills

Designing a user-friendly interface helped improve UI/UX development skills.

Skills Learned:

- Layout design
- Responsive UI design
- Navigation structure design
- User-centered design principles

Benefits:

Improved ability to design clean and intuitive interfaces.

Debugging and Problem-Solving Skills

Several technical challenges helped improve debugging skills.

Skills Learned:

- Error identification
- Console debugging
- API troubleshooting
- Performance issue resolution

Benefits:

Enhanced analytical thinking and software debugging capability.

Software Testing Skills

The project required systematic testing of all modules.

Skills Learned:

- Functional testing
- UI testing
- API testing
- Browser compatibility testing

Benefits:

Improved understanding of software quality assurance practices.

Development Tools Usage

Practical exposure to development tools improved productivity.

Tools Used:

- Visual Studio Code
- Chrome Developer Tools
- React Developer Tools
- Vite Development Server
- Git & GitHub

Benefits:

Improved development efficiency and workflow management.

Project Documentation Skills

Preparing documentation improved technical writing abilities.

Skills Learned:

- Technical report writing
- System documentation
- Architecture documentation
- Testing documentation

Benefits:

Enhanced professional communication and documentation skills.

Project Management Skills

Managing the project improved planning and organizational abilities.

Skills Learned:

- Requirement analysis
- Task scheduling
- Time management
- Workflow organization

Benefits:

Improved ability to manage software development projects effectively.

Teamwork and Communication Skills

Collaboration during development improved communication skills.

Skills Learned:

- Team collaboration
- Technical communication
- Problem discussion
- Knowledge sharing

Benefits:

Strengthened interpersonal and professional communication abilities.

Summary of Skills Acquired

- ReactJS Development
 - JavaScript Programming
 - API Integration
 - Leaflet Maps Implementation
 - OpenStreetMap Usage
 - Responsive Web Design
 - UI/UX Design Principles
 - Debugging and Troubleshooting
 - Software Testing Techniques
 - Git & GitHub Version Control
 - Technical Documentation
 - Project Management
 - Teamwork and Communication
 - Problem-Solving Skills
-

Conclusion

The development of the **Flight Status Tracker System** significantly enhanced both technical and professional skills. The project provided practical experience in ReactJS development, API integration, real-time data handling, testing, debugging, and documentation. These skills will be valuable for future academic projects and professional software development careers.

CHAPTER -13 REFERENCES

The development of the **Flight Status Tracker System** involved the use of various technologies, frameworks, libraries, documentation resources, and online references. These resources provided guidance for system design, implementation, API integration, testing, deployment, and flight data visualization functionalities.

Books

- [1] Roger S. Pressman and Bruce R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th Edition, McGraw-Hill Education, 2020.
 - [2] Ian Sommerville, *Software Engineering*, 10th Edition, Pearson Education, 2015.
 - [3] Jon Duckett, *JavaScript and JQuery: Interactive Front-End Web Development*, Wiley Publications, 2014.
 - [4] David Flanagan, *JavaScript: The Definitive Guide*, 7th Edition, O'Reilly Media, 2020.
-

Online Documentation

- [5] ReactJS Official Documentation. Available: <https://react.dev>
 - [6] Vite Official Documentation. Available: <https://vitejs.dev>
 - [7] Leaflet Official Documentation. Available: <https://leafletjs.com>
 - [8] OpenStreetMap Documentation. Available: <https://www.openstreetmap.org>
 - [9] Mozilla Developer Network (MDN) Web Docs. Available: <https://developer.mozilla.org>
 - [10] Geolocation API Documentation. Available: https://developer.mozilla.org/en-US/docs/Web/API/Geolocation_API
-

Development Tools

- [11] Visual Studio Code Documentation. Available: <https://code.visualstudio.com>
 - [12] Git Documentation. Available: <https://git-scm.com/doc>
 - [13] GitHub Documentation. Available: <https://docs.github.com>
 - [14] Chrome Developer Tools Documentation. Available: <https://developer.chrome.com/docs/devtools>
-

Flight Data and Mapping Resources

- [15] Flight Data API Documentation (used for real-time flight information integration).
 - [16] Leaflet Mapping Library Documentation. Available: <https://leafletjs.com/reference.html>
 - [17] OpenStreetMap Foundation Resources. Available: <https://www.openstreetmap.org>
 - [18] Browser Geolocation API Documentation. Available: https://developer.mozilla.org/en-US/docs/Web/API/Geolocation_API
-

Software and Technologies Used

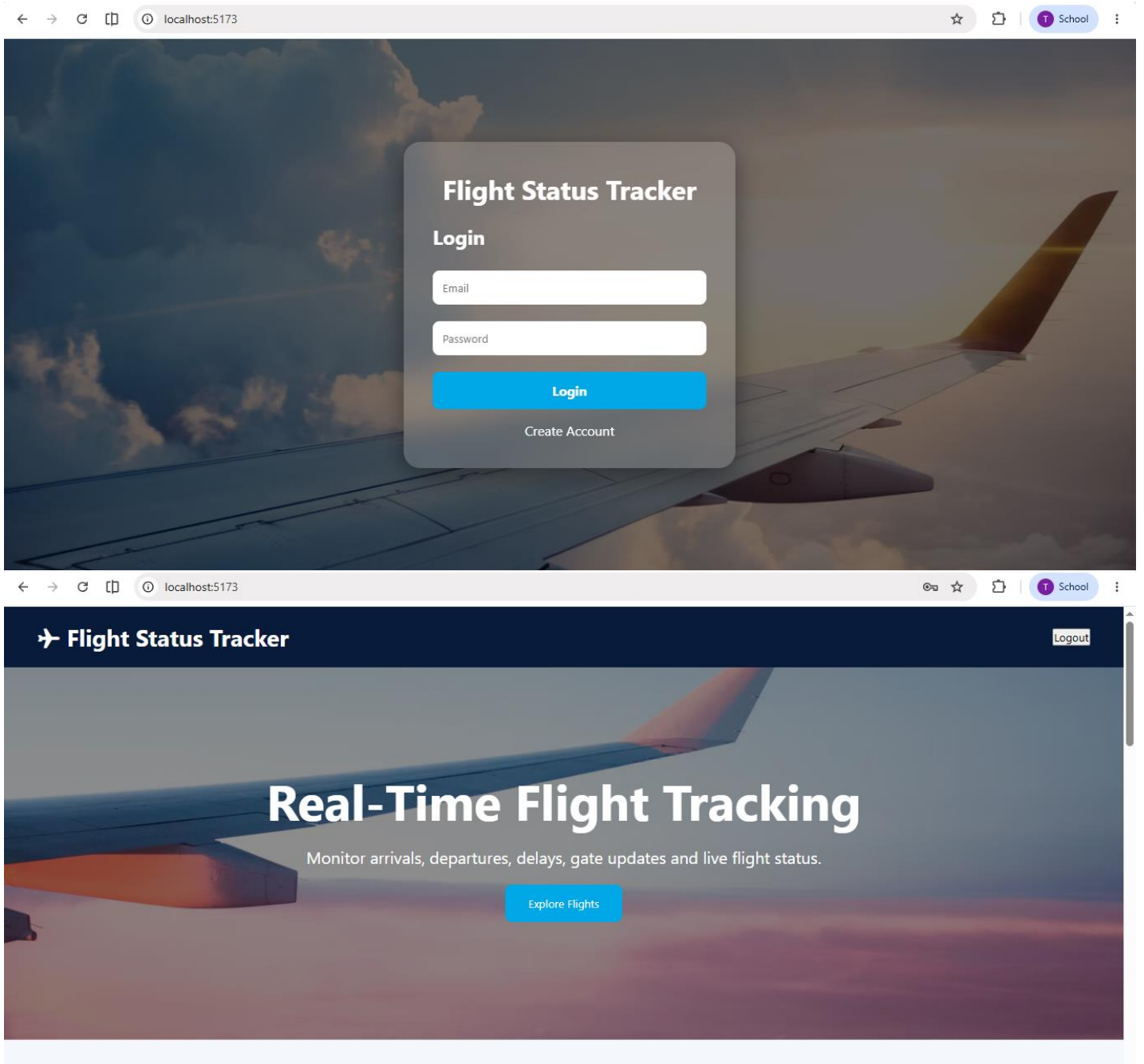
- [19] ReactJS Frontend Framework
 - [20] JavaScript Programming Language
 - [21] HTML5 and CSS3
 - [22] Vite Development Environment
 - [23] Leaflet Interactive Mapping Library
 - [24] OpenStreetMap Mapping Services
 - [25] Flight Data APIs for real-time flight tracking
-

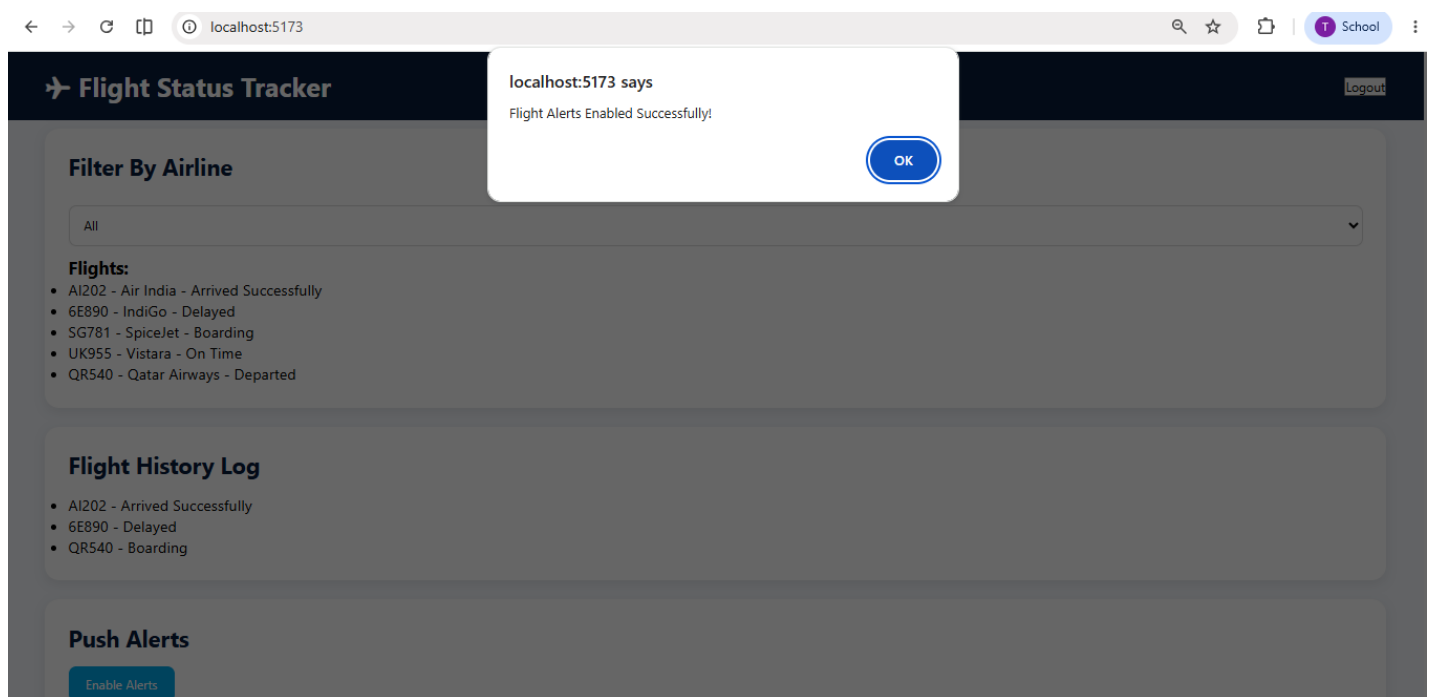
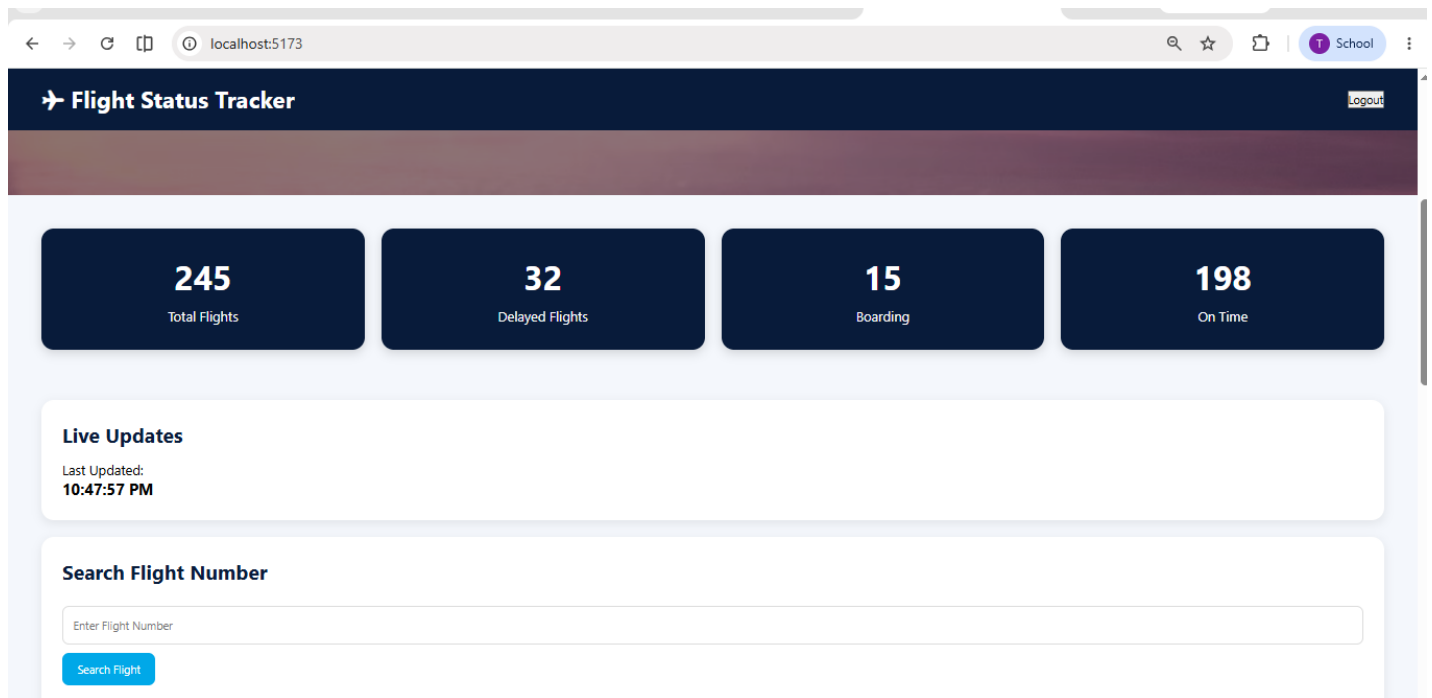
Conclusion

The above references provided the theoretical knowledge, technical guidance, development support, and implementation resources necessary for the successful development of the **Flight Status Tracker System**. These references were extensively used during system design, API integration, coding, testing, deployment, and documentation preparation.

CHAPTER -14 APPENDICES

Screenshots of the Website



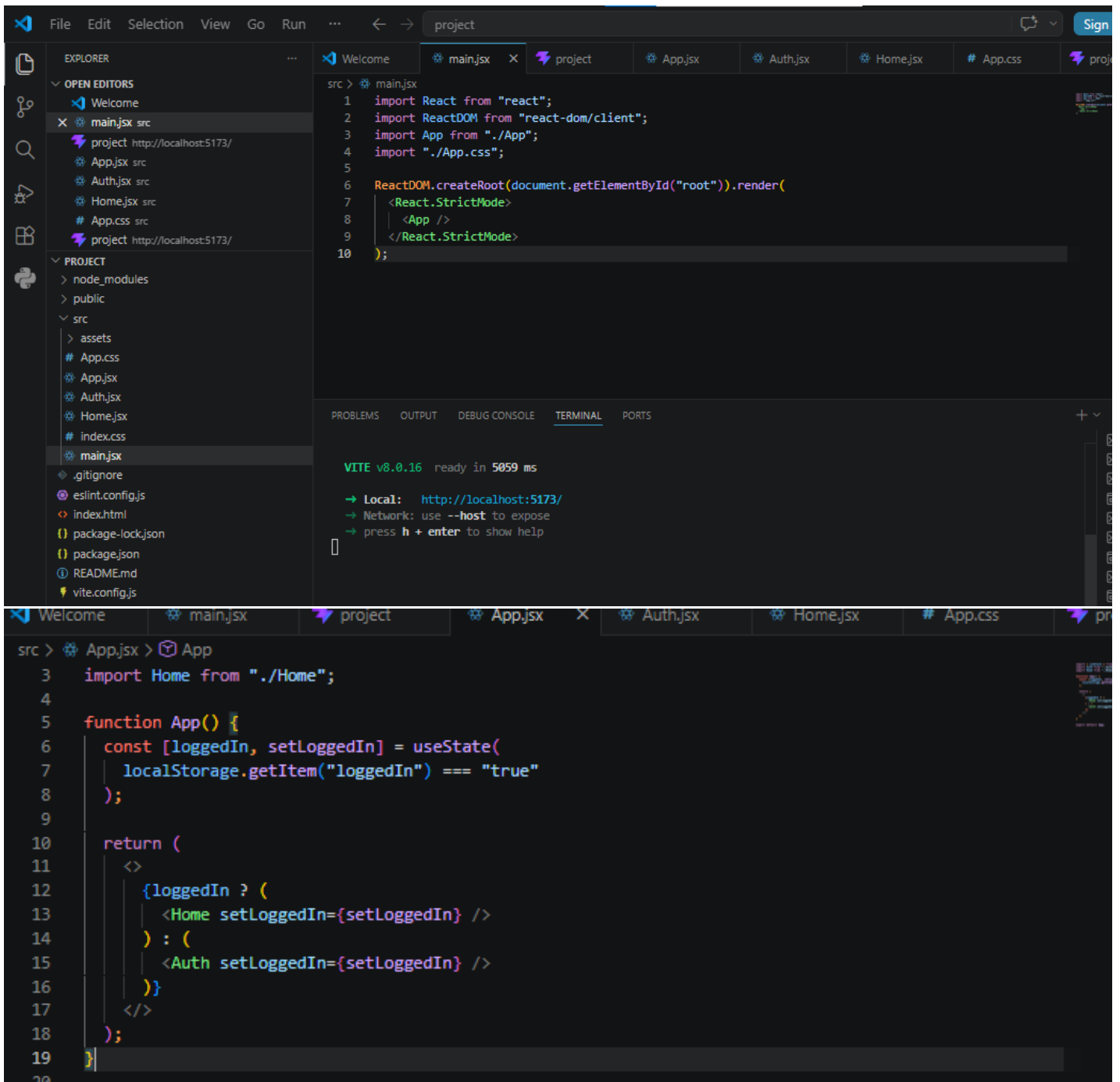


[Open in Maps](#)



Delete Flight

Sample code snippets



The image shows a VS Code editor with a React application. The Explorer sidebar on the left shows the project structure:

- EXPLORED
 - OPEN EDITORS
 - Welcome
 - main.jsx src
 - project http://localhost:5173/
 - App.jsx src
 - Auth.jsx src
 - Home.jsx src
 - App.css src
 - PROJECT
 - node_modules
 - public
 - src
 - assets
 - App.css
 - App.jsx
 - Auth.jsx
 - Home.jsx
 - index.css
 - main.jsx
 - .gitignore
 - eslint.config.js
 - index.html
 - package-lock.json
 - package.json
 - README.md
 - vite.config.js

The main editor shows the following code in `main.jsx`:

```
src > main.jsx
1 import React from "react";
2 import ReactDOM from "react-dom/client";
3 import App from "./App";
4 import "./App.css";
5
6 ReactDOM.createRoot(document.getElementById("root")).render(
7   <React.StrictMode>
8     <App />
9   </React.StrictMode>
10 );
```

The terminal at the bottom shows the following output:

```
VITE v8.0.16 ready in 5059 ms
→ Local: http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

The Explorer sidebar also shows the following code in `App.jsx`:

```
src > App.jsx > App
3 import Home from "./Home";
4
5 function App() {
6   const [loggedIn, setLoggedIn] = useState(
7     localStorage.getItem("loggedIn") === "true"
8   );
9
10  return (
11    <>
12      {loggedIn ? (
13        <Home setLoggedIn={setLoggedIn} />
14      ) : (
15        <Auth setLoggedIn={setLoggedIn} />
16      )}
17    </>
18  );
19 }
```

```

1 import { useState } from "react";
2
3 function Auth({ setLoggedIn }) {
4   const [isLoggedIn, setLoggedIn] = useState(true);
5
6   const [formData, setFormData] = useState({
7     username: "",
8     email: "",
9     password: "",
10  });
11
12  const handleChange = (e) => {
13    setFormData({
14      ...formData,
15      [e.target.name]: e.target.value,
16    });
17  };
18
19  const handleSubmit = (e) => {
20    e.preventDefault();
21    localStorage.setItem("loggedIn", "true");
22    localStorage.setItem("username", formData.username);
23
24    setLoggedIn(true);
25  };
26
27  return (
28    <div className="auth-container">
29      <form className="auth-form" onSubmit={handleSubmit}>
30        <h1>Flight Status Tracker</h1>
31
32        <h2>{isLoggedIn ? "Login" : "Signup"}</h2>
33
34        {isLoggedIn && (
35          <input
36            type="text"
37            name="username"
38            placeholder="Username"
39            required
40            onChange={handleChange}
41          />
42        )}
43
44        <input
45          type="email"
46          name="email"
47          placeholder="Email"
48          required
49          onChange={handleChange}
50        />
51
52        <input
53          type="password"
54          name="password"
55          placeholder="Password"
56          required
57          onChange={handleChange}
58        />
59
60        <button type="submit">
61          {isLoggedIn ? "Login" : "Signup"}
62        </button>
63
64        <input
65          type="password"
66          name="password"
67          placeholder="Password"
68          required
69          onChange={handleChange}
70        />
71
72        <button type="submit">
73          {isLoggedIn ? "Login" : "Signup"}
74        </button>
75
76        <div
77          className="switch"
78          onClick={() => setLoggedIn(!isLoggedIn)}
79        >
80          {isLoggedIn
81            ? "Create Account"
82            : "Already have an account?"}
83        </div>
84      </form>
85    </div>
86  );
87 }

```

c > Home.jsx > Home > flights

```
1  import { useState, useEffect } from "react";
2
3  function Home({ setLoggedIn }) {
4    const [flightNo, setFlightNo] = useState("");
5    const [airline, setAirline] = useState("All");
6    const [time, setTime] = useState(new Date());
7
8    const [result, setResult] = useState("");
9    const [message, setMessage] = useState("");
10
11    const flights = [
12      {
13        number: "AI202",
14        airline: "Air India",
15        status: "Arrived Successfully",
16      },
17      {
18        number: "6E890",
19        airline: "IndiGo",
20        status: "Delayed",
21      },
22      {
23        number: "SG781",
24        airline: "SpiceJet",
25        status: "Boarding",
26      },
27    ],
```

rc > # App.css > .hero button

```
1  *{
2    margin:0;
3    padding:0;
4    box-sizing:border-box;
5    font-family:'Segoe UI',sans-serif;
6  }
7
8  body{
9    background: #f4f7fc;
10 }
11
12 /* LOGIN PAGE */
13
14 .auth-container{
15   min-height:100vh;
16   display:flex;
17   justify-content:center;
18   align-items:center;
19
20   background:
21     linear-gradient(
22       rgba(0,0,0,0.55),
23       rgba(0,0,0,0.55)
24     ),
25     url("https://images.unsplash.com/photo-1436491865332-7a61a109cc05?auto=format&fit=crop&w=1400&q=80");
26
27   background-size:cover;
```

Installation / Setup Instructions

The Flight Status Tracker is developed using **ReactJS and Vite**. The application can be installed and executed on any system with Node.js and a modern web browser.

Steps:

1. Download or extract the project source code.
2. Open the project folder in **Visual Studio Code**.
3. Open the terminal inside the project directory.
4. Install dependencies using:

```
npm install
```

5. Start the development server using:

```
npm run dev
```

6. Open the browser and go to:

```
http://localhost:5173
```

7. The application will launch successfully and display the **Flight Status Tracker dashboard**.
-

User Manual / Guide

The Flight Status Tracker allows users to search, view, and monitor real-time flight information in a simple web interface.

Steps to Use:

1. Open the application in a web browser.
2. Login using the authentication page.
3. Enter a **flight number** in the search box.
4. View flight details such as airline and status.
5. Use filters to view flights by airline.
6. Check live updates and flight statistics dashboard.
7. Explore flight history logs for previous records.
8. Enable alerts for important flight updates.
9. Subscribe to notifications for real-time updates.

Geo Tag Photo



Poster:

Flight Status Tracker Management

Project based learning -FEDE



KLUH
KANNUR UNIVERSITY
BACHUPALLY CAMPUS

ABSTRACT

The Flight Status Tracker is a web-based application that provides real-time flight information to passengers and airline personnel. The system allows users to search for flights using flight numbers and view important details such as departure and arrival times, gate information, and flight status. It integrates REST APIs to fetch live flight data and uses polling techniques to ensure updates are displayed in real time. Users can also subscribe to notifications for selected flights and receive alerts about delays or schedule changes. The application is developed using React.js, Node.js with optional MySQL integration for storing historical flight data and user preferences. Additional features include a flight status dashboard, airline-based filtering, map view, responsive design, and an admin panel. Overall, the Flight Status Tracker provides a reliable and user-friendly platform for monitoring flight information efficiently.

SYSTEM ARCHITECTURE



The diagram illustrates the system architecture, divided into Existing System Architecture and Proposed System Architecture. The Existing System Architecture shows a flow from User to API Gateway, then to Backend Services, and finally to Frontend Services. The Proposed System Architecture shows a more complex flow involving User, API Gateway, Backend Services, and Frontend Services, with additional components like Database and Authentication. The diagram also includes a list of features and a list of technologies used.

CO-WISE IMPLEMENTATION

CO-1 Component-Based Architecture

```
import Auth from './Auth';
import Home from './Home';

function App() {
  return (
    <Auth />
    <Home />
  );
}

export default App;
```

Declarative UI
Reactive State Management
import { useState } from 'react';

CO2-ES6 Functional Programming

```
const add = (a, b) => a + b;
const multiply = (a, b) => a * b;
const divide = (a, b) => a / b;
const subtract = (a, b) => a - b;
const calculate = (a, b, operation) => operation(a, b);
const operations = {
  add,
  multiply,
  divide,
  subtract,
};
const result = calculate(10, 5, operations.add);
console.log(result);
```

CO3-React Component Model

```
import React, { useState } from 'react';
import './App.css';

function App() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h1>React Component Model</h1>
      <button onClick={() => setCount(count + 1)}>
        Click me
      </button>
      <p>Count: {count}</p>
    </div>
  );
}

export default App;
```

CO4-State Architecture, Async Data Engineering & API Integration

```
import React, { useState, useEffect } from 'react';
import './App.css';

function App() {
  const [data, setData] = useState(null);

  useEffect(() => {
    fetch('https://api.example.com/data')
      .then(response => response.json())
      .then(data => setData(data));
  }, []);

  return (
    <div>
      <h1>State Architecture, Async Data Engineering & API Integration</h1>
      <p>Data: {data}</p>
    </div>
  );
}

export default App;
```

CO5-Router Forms, Accessibility,

```
import React, { useState } from 'react';
import './App.css';

function App() {
  const [username, setUsername] = useState('');
  const [password, setPassword] = useState('');
  const [login, setLogin] = useState(false);

  return (
    <div>
      <h1>Router Forms, Accessibility,</h1>
      <input type="text" value={username} />
      <input type="password" value={password} />
      <button onClick={() => setLogin(true)}>
        Login
      </button>
      <p>Login: {login}</p>
    </div>
  );
}

export default App;
```

CO6-React Entry

```
import React, { useState } from 'react';
import './App.css';

function App() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h1>React Entry</h1>
      <button onClick={() => setCount(count + 1)}>
        Click me
      </button>
      <p>Count: {count}</p>
    </div>
  );
}

export default App;
```

React Strict Mode

CONCLUSION

The Flight Status Tracker is a user-friendly web application that enables users to monitor flight information quickly and efficiently. Developed using React.js, it provides features such as flight search, airline filtering, live status updates, notifications, and a responsive interface. The project demonstrates the effective use of modern web technologies to improve access to flight information and provides a strong foundation for future enhancements such as real-time API integration, database support, and advanced tracking features.

Name : Thunga Sree Chandana,
Naralasetty Hasini
Roll No : 2520040030,2520040039
Section : 15
Cluster : 5